

使用 ADK Visual Builder 建立及部署低程式碼 ADK (Agent Deployment Kit) 代理

來源：<https://codelabs.developers.google.com/codelabs/create-low-code-agent-with-ADK-visual-builder?hl=zh-tw>

步驟數：13

在本實驗室中，您將使用 ADK Visual Builder 建立低程式碼 ADK 代理

目錄

1. 本實驗室的目標
2. 專案設定
3. 啟用 API
4. 確認是否已套用抵免額
5. Agent Development Kit 簡介
6. 安裝 ADK 並設定環境
7. 使用 ADK Visual Builder 建立簡單的代理
8. 將 Agent 部署至 Cloud Run
9. 建立具有子代理和自訂工具的代理
10. 建立工作流程代理程式
11. 使用 Agent Builder 助理建立代理
12. 清除所用資源
13. 結論

步驟 1 · 約 0 分鐘

1. 本實驗室的目標

在本實作實驗室中，您將瞭解如何使用 [ADK \(Agent Development Kit\) Visual Builder](#) 建立代理。 [ADK \(Agent Development Kit\) Visual Builder](#) 提供低程式碼方式，可建立 [ADK \(Agent Development Kit\)](#) 代理。您將瞭解如何在本機測試應用程式，並部署至 [Cloud Run](#)。

課程內容

- 瞭解 [ADK \(Agent Development Kit\)](#) 的基礎知識。
- 瞭解 [ADK \(Agent Development Kit\) Visual Builder](#) 的基礎知識
- 瞭解如何使用 GUI 工具建立代理程式。
- 瞭解如何在 [Cloud Run](#) 中輕鬆部署及使用代理程式。



圖 1：使用 ADK Visual Builder，您可以使用低程式碼的 GUI 建立代理

2. 專案設定

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

- 如果沒有可用的專案，請在 [GCP 主控台](#) 中建立新專案。在專案選取器 (Google Cloud 控制台左上角) 中選取專案

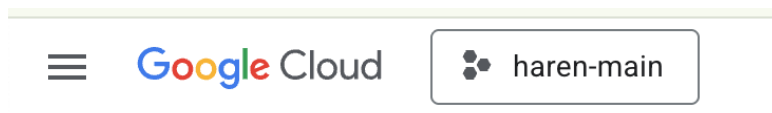


圖 2：按一下 Google Cloud 標誌旁的方塊，即可選取專案。確認已選取專案。

- 在本實驗室中，我們將使用 [Cloud Shell 編輯器](#) 執行工作。開啟 Cloud Shell，並使用 Cloud Shell 設定專案。
- 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
- 如果尚未開啟終端機，請依序點選選單中的「Terminal」>「New Terminal」。您可以在這個終端機中執行本教學課程的所有指令。
- 您可以在 Cloud Shell 終端機中執行下列指令，檢查專案是否已通過驗證。

〇〇〇〇

```
gcloud auth list
```

- 在 Cloud Shell 中執行下列指令，確認專案

〇〇〇〇

```
gcloud config list project
```

- 複製專案 ID，然後使用下列指令設定

〇〇〇〇

```
gcloud config set project <YOUR_PROJECT_ID>
```

- 如果忘記專案 ID，可以使用下列指令列出所有專案 ID：

gcloud projects list

步驟 3 · 約 0 分鐘

3. 啟用 API

我們需要啟用一些 API 服務，才能執行本實驗室。在 Cloud Shell 中執行下列指令。

```
gcloud services enable aiplatform.googleapis.com
gcloud services enable cloudresourcemanager.googleapis.com
```

API 簡介

- [Vertex AI](#) API (`aiplatform.googleapis.com`) 可存取 [Vertex AI](#) 平台，讓應用程式與 Gemini 模型互動，生成文字、進行聊天工作階段及呼叫函式。
 - Cloud Resource Manager API (`cloudresourcemanager.googleapis.com`) 可讓您以程式輔助方式管理 Google Cloud 專案的中繼資料，例如專案 ID 和名稱。其他工具和 SDK 通常需要這些資料，才能驗證專案身分和權限。
-

步驟 4 · 約 0 分鐘

4. 確認是否已套用抵免額

在「專案設定」階段，您已申請免費抵免額，可使用 Google Cloud 服務。套用抵免額後，系統會建立名為「Google Cloud Platform Trial Billing Account」的新免費帳單帳戶。如要確認抵免額已套用，請在 [Cloud Shell 編輯器](#) 中按照下列步驟操作：

```
curl -s https://raw.githubusercontent.com/haren-bh/gcpbillingactivate/main/activate.py |  
python3
```

如果成功，您應該會看到如下的結果：如果看到「Successfully linked project」（專案連結成功），表示帳單帳戶設定正確。執行上述步驟後，系統會檢查你的帳戶是否已連結，如果沒有，系統會為你連結。如果您尚未選取專案，系統會提示您選擇專案，您也可以事先按照專案設定中的步驟操作。

```
Finding an active billing account...  
Found billing account: Google Cloud Platform Trial Billing Account (014890-08C583-49ACEE)  
Finding current project...  
Using project: testproject1-482302  
Linking project testproject1-482302 to billing account Google Cloud Platform Trial Billing Account (014890-08C583-49ACEE)...  
Successfully linked project.
```

圖 3：確認已連結帳單帳戶

5. Agent Development Kit 簡介

Agent Development Kit 為需要建構代理應用程式的開發人員帶來多項關鍵優勢：

- 1. **多代理系統**：可在階層結構中組合多個專用代理，建構出可擴充的模組化應用程式，藉以執行複雜的協調和委派作業。
- 2. **豐富的工具生態系統**：為代理提供多元功能，像是使用預先建構的工具 (搜尋、程式碼執行等)、建立自訂函式、整合第三方代理框架的工具 (LangChain、CrewAI)，甚至還可以使用其他代理做為工具。
- 3. **彈性的自動化調度管理功能**：使用工作流程代理 (SequentialAgent 、 ParallelAgent 和 LoopAgent) 定義可預測的管道工作流程，或透過使用 LLM 的動態轉送功能 (LlmAgent 轉移)，創造靈活應變的代理。
- 4. **整合式開發人員體驗**：使用功能強大的 CLI 和互動式開發 UI，在本機開發、測試及偵錯，逐步檢查事件、狀態和代理執行作業。
- 5. **內建評估功能**：與預先定義的測試案例比較，評估最終回覆品質和逐步執行軌跡，有系統地判斷代理效能。
- 6. **可隨時部署**：將代理容器化並部署至任何位置，無論是在本機執行、透過 Vertex AI Agent Engine 擴充，或使用 Cloud Run/Docker 整合至自訂基礎架構都沒問題。

雖然其他生成式 AI SDK 或代理框架同樣能查詢模型，甚至為模型提供工具，但您必須投入大量心力，在多個模型間靈活調度。

相較之下，Agent Development Kit 比上述工具更加高階，您可以輕鬆相互連結多個代理，建立複雜但方便維護的工作流程。

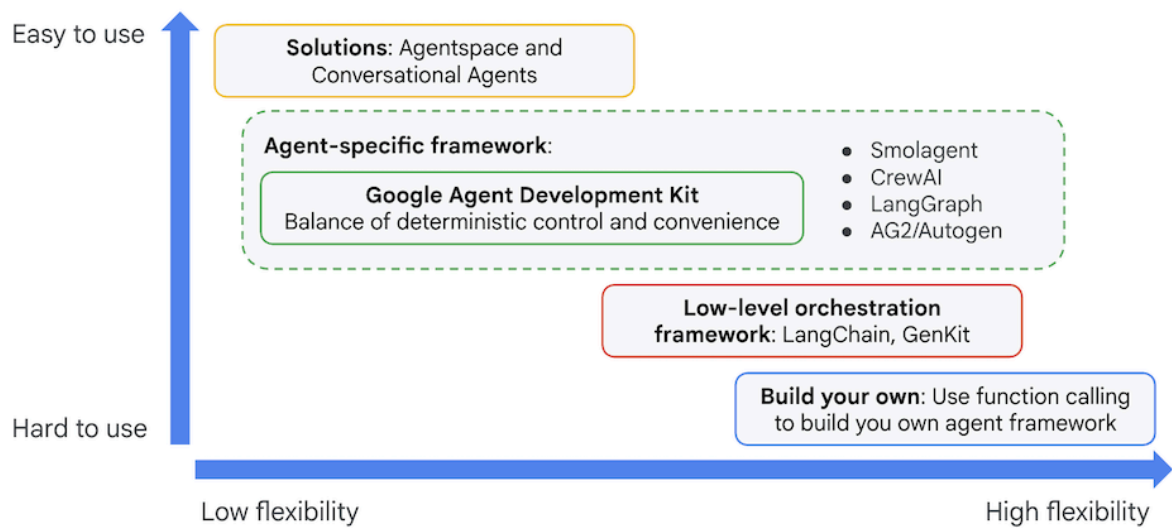


圖 4：ADK (Agent Development Kit) 的定位

在最新版本中，ADK (Agent Development Kit) 新增了 ADK Visual Builder 工具，可讓您以低程式碼方式建構 ADK (Agent Development Kit) 代理。在本實驗室中，我們將詳細瞭解 ADK Visual Builder 工具。

步驟 6 · 約 0 分鐘

6. 安裝 ADK 並設定環境

首先，我們需要設定環境，才能執行 [ADK \(Agent Development Kit\)](#)。在本實驗室中，我們將執行 [ADK \(Agent Development Kit\)](#)，並在 Google Cloud 的 [Cloud Shell 編輯器](#) 中執行所有工作。

準備 Cloud Shell 編輯器

1. 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
2. 按一下「繼續」。
3. 系統提示您授權 Cloud Shell 時，點選「授權」。
4. 操作本實驗室其餘步驟時，您可以全程將這個視窗當成 IDE 使用，搭配 [Cloud Shell 編輯器](#) 和 Cloud Shell 終端機作業。
5. 在 Cloud Shell 編輯器中，依序點選「Terminal」(終端機) > 「New Terminal」(新增終端機)，開啟新的終端機。下列所有指令都會在這個終端機上執行。

啟動 ADK 視覺化編輯器

1. 執行下列指令，從 GitHub 複製所需來源，並安裝必要程式庫。在 [Cloud Shell 編輯器](#) 中開啟終端機，然後執行指令。

```
#create the project directory
mkdir ~/adkui
cd ~/adkui
```

2. 我們會使用 uv 建立 Python 環境 (在 [Cloud Shell 編輯器](#) 終端機中執行)：

```
#Install uv if you do not have installed yet
pip install uv

#go to the project directory
cd ~/adkui

#Create the virtual environment
uv venv

#use the newly created environment
source .venv/bin/activate

#install libraries
uv pip install google-adk==1.22.1
uv pip install python-dotenv
```

注意：如果需要重新啟動終端機，請務必執行「`source .venv/bin/activate`」設定 Python 環境

3. 在編輯器中，依序前往「View」->「Toggle hidden files」。在 `adkui` 資料夾中，建立含有以下內容的 `.env` 檔案。

```
#go to adkui folder
cd ~/adkui
```

```
cat <<EOF>> .env
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project)
GOOGLE_CLOUD_LOCATION=us-central1
IMAGEN_MODEL="imagen-3.0-generate-002"
GENAI_MODEL="gemini-2.5-flash"
EOF
```

步驟 7 · 約 0 分鐘

7. 使用 ADK Visual Builder 建立簡單的代理

在本節中，我們將使用 [ADK Visual Builder](#) 建立簡單的代理程式。[ADK Visual Builder](#) 是一種網頁工具，提供視覺化工作流程設計環境，用於建立及管理 [ADK \(Agent Development Kit\)](#) 代理程式。您可以在簡單易用的圖形介面中設計、建構及測試代理程式，還能使用 AI 輔助建構代理程式。

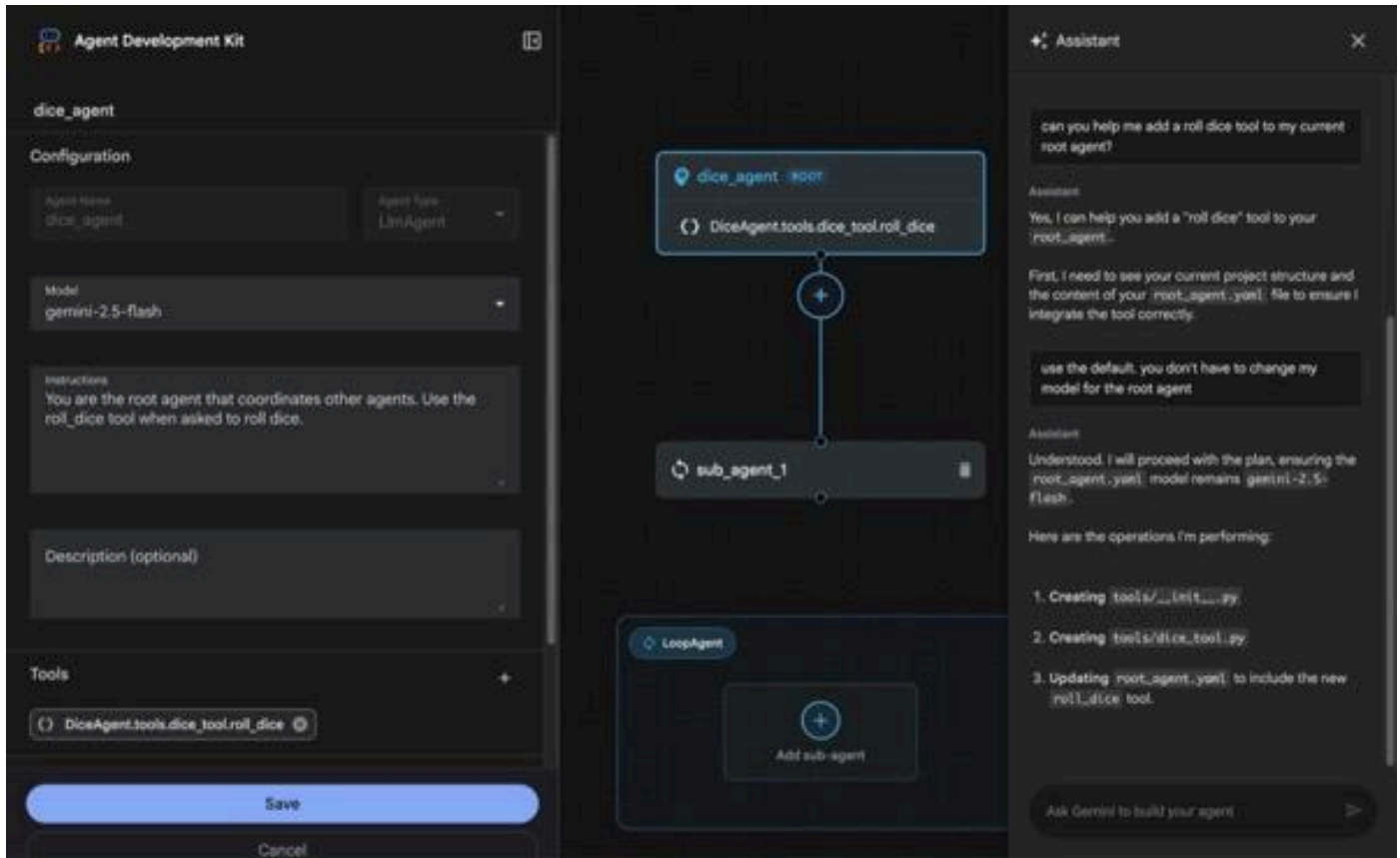


圖 5：ADK Visual Builder

重要事項：關於 [ADK Visual Builder](#) (本實驗室發布時)

[ADK Visual Builder](#) 可讓您使用 GUI 介面，以少量程式碼建立代理。使用 [ADK Visual Builder](#) 建立代理程式時，系統會將 yaml 檔案或工具等構件儲存在 tmp 資料夾中。你必須按下「儲存」，才能將這項設定保留在主要代理程式資料夾中。如果您在 tmp 資料夾中進行變更，請務必返回 GUI 並點選「儲存」，才能保留變更。|

1. 返回終端機中的頂層目錄 **adkui**，然後執行下列指令，在本機執行代理程式 (在 [Cloud Shell 編輯器](#) 終端機中執行)。您應該可以啟動 ADK 伺服器，並在終端機中看到類似圖 6 的結果。

```
#go to the directory adkui
cd ~/adkui
# Run the following command to run ADK locally
adk web
```

```
INFO: Started server process [1875]
INFO: Waiting for application startup.

+-----+
| ADK Web Server started |
|                         |
| For local testing, access at http://localhost:8000. |
+-----+

INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

圖 6：啟動 ADK 應用程式

2. 在終端機上顯示的 `http://` URL 上按住 **Ctrl** 鍵並點選 (在 MacOS 上按住 **CMD** 鍵並點選)，即可開啟 **ADK (Agent Development Kit)** 瀏覽器型 GUI 工具。

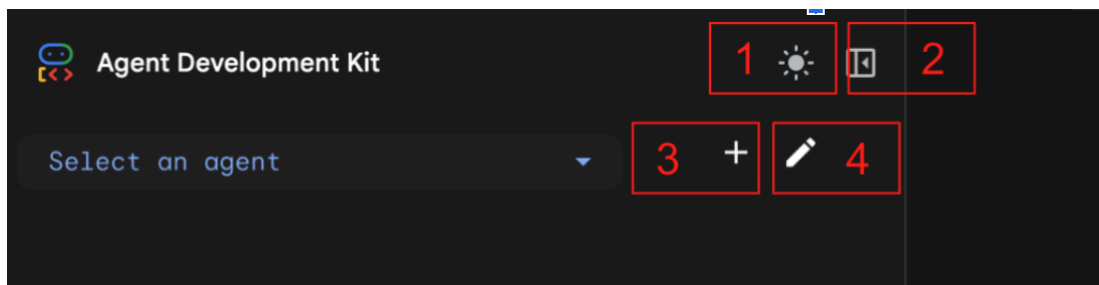


圖 7：ADK 網頁式 UI，ADK 包含下列元件：1：切換淺色和深色模式 2：收合面板 3：建立代理 4：編輯代理

3. 如要建立新的代理程式，請按下「+」按鈕。

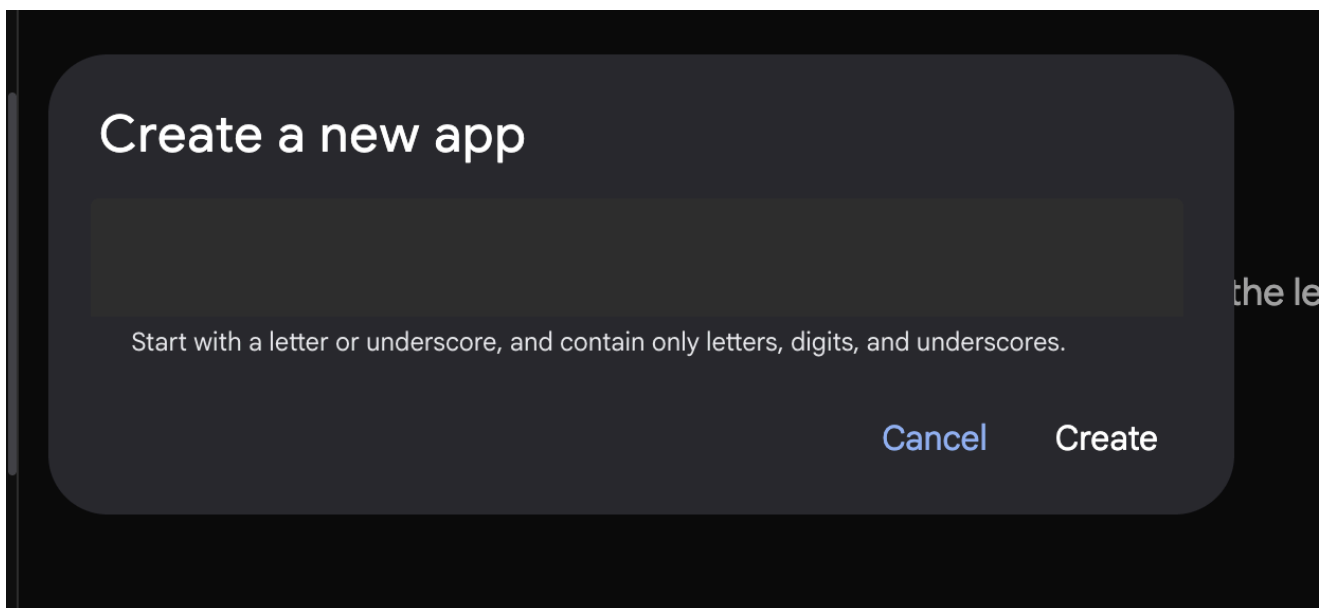


圖 8：建立新應用程式的對話方塊

4. 命名為「Agent1」，然後按一下「建立」。

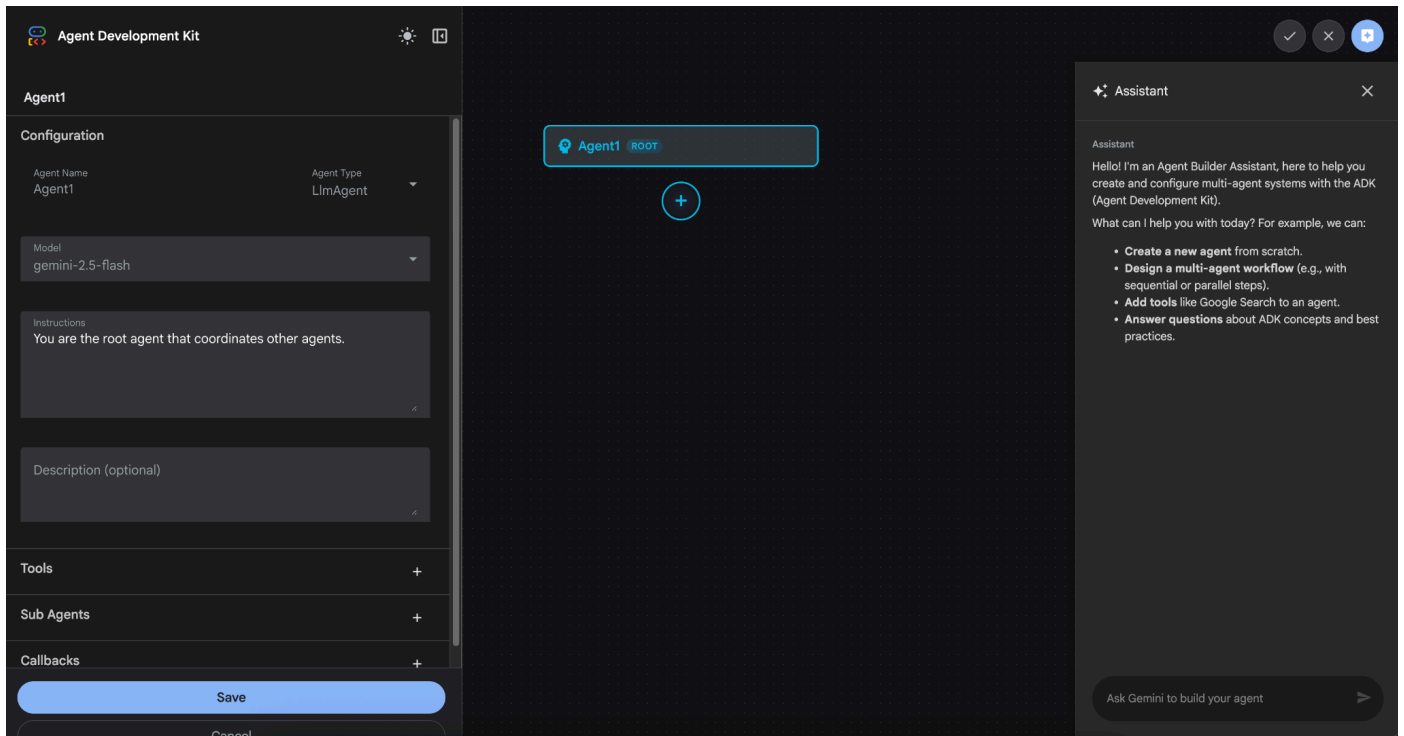


圖 9：代理程式建構工具的 UI

4. 面板分為三個主要部分：左側是 GUI 型代理程式建立的控制項，中間是進度視覺化畫面，右側則是使用自然語言建立代理程式的助理。
5. 代理已成功建立。按一下「儲存」按鈕繼續操作。(注意：請務必按下「儲存」，以免遺失變更。)
6. 現在應該可以測試服務專員了。首先，請在「即時通訊」方塊中輸入提示，例如：

Hi, what can you do?

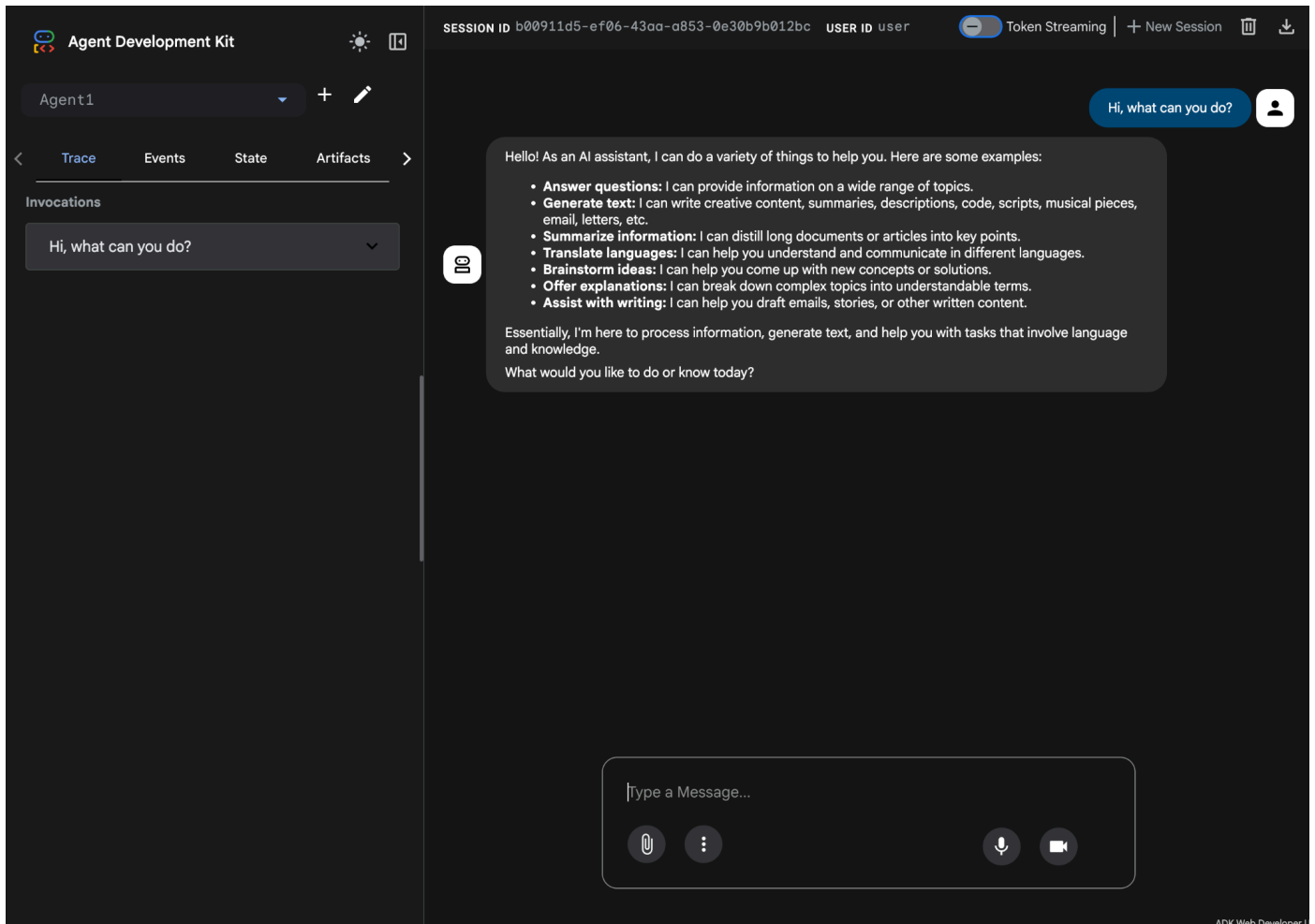


圖 10：測試代理。

7. 返回編輯器，檢查新產生的檔案。您會在左側看到檔案總管。前往 `adkgui` 資料夾並展開，顯示 `Agent 1` 目錄。在資料夾中，您可以查看定義代理程式的 `YAML` 檔案，如下圖所示。

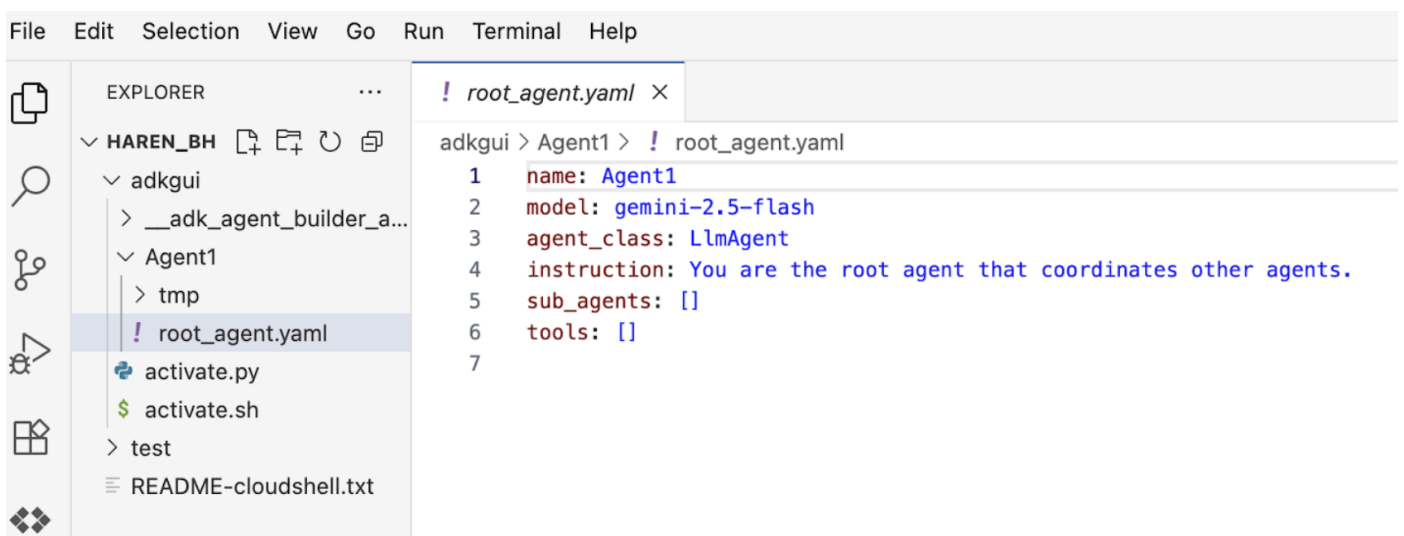


圖 11：使用 `YAML` 檔案定義代理

- 現在返回 **GUI 編輯器**，為代理程式新增幾項功能。如要編輯，請按下**編輯按鈕** (請參閱圖 7，元件編號 4，筆圖示)。
- 我們將為代理新增 **Google 搜尋**功能，因此需要將 `Google 搜尋`新增為代理可用的工具，以及代理可使用的工具。如要這麼做，請按一下畫面左下角「工具」部分旁邊的「+」符號，然後從選單中點選「內建工具」 (請參閱圖 12)。

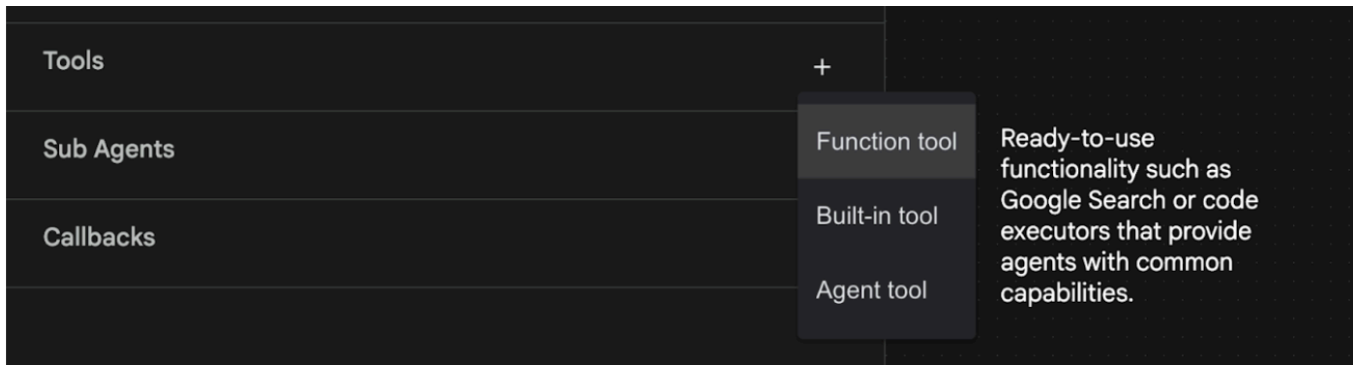


圖 12：將新工具新增至代理程式

10. 從「內建工具」清單中選取「google_search」，然後按一下「建立」(請參閱圖 12)。這會將 Google 搜尋新增為代理程式的工具。
11. 按下「儲存」按鈕，儲存變更。

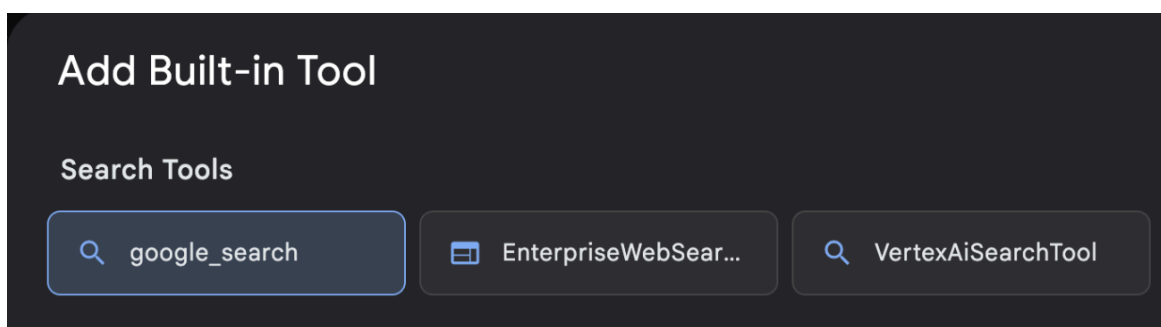


圖 13：ADK Visual Builder UI 中可用的工具清單

11. 現在可以測試 Agent 了。請先重新啟動 ADK 伺服器。前往啟動 [ADK \(Agent Development Kit\)](#) 伺服器的終端機，然後按下 **CTRL+C** 鍵關閉伺服器 (如果伺服器仍在執行)。執行下列指令，再次啟動伺服器。

```
#make sure you are in the right folder.
cd ~/adkui

#start the server
adk web
```

Note:

In case you get an error saying port 8000 is already in use, use the following command to kill the service and restart it (Run step 14).

```
ss -lntp | grep ":8000" | awk '{print $NF}' | grep -oP 'pid=\K\d+' | xargs -r kill -9
```

12. 按住 Ctrl 鍵並點選網址 (例如 <http://localhost:8000>) 顯示在畫面上。瀏覽器分頁應會顯示 [ADK \(Agent Development Kit\)](#) GUI。
13. 從服務專員清單中選取「Agent1」 **Agent1**。現在，代理程式可以執行 **Google 搜尋**。在對話方塊中，使用下列提示詞進行測試。

```
What is the weather today in Yokohama?
```


畫面應會顯示 Google 搜尋結果，如下所示。

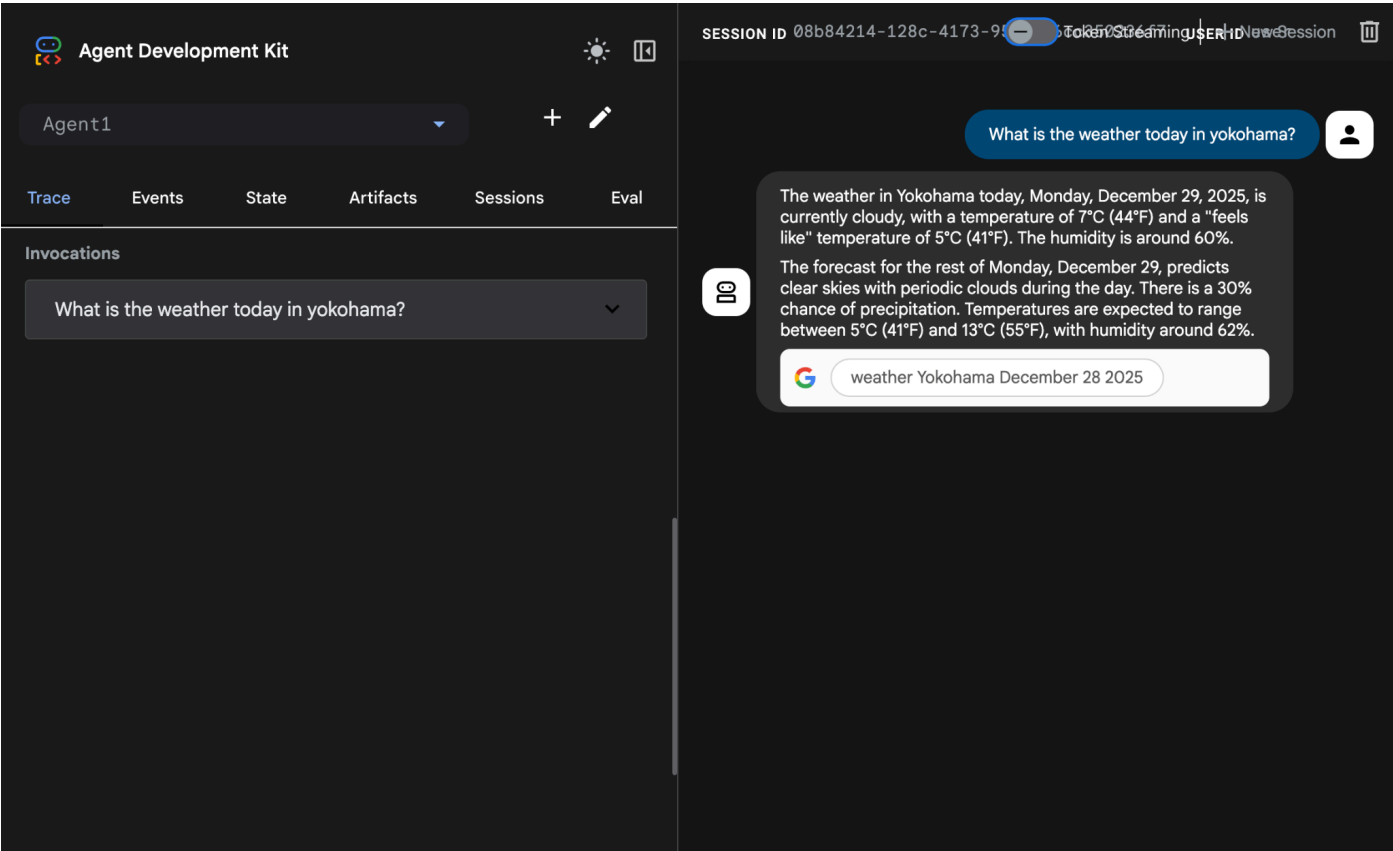


圖 14：使用代理執行 Google 搜尋

12. 現在讓我們回到編輯器，檢查這個步驟中建立的程式碼。在編輯器「Explorer」側邊面板中，按一下「root_agent.yaml」開啟檔案。確認 **google_search** 已新增為工具 (圖 15)。

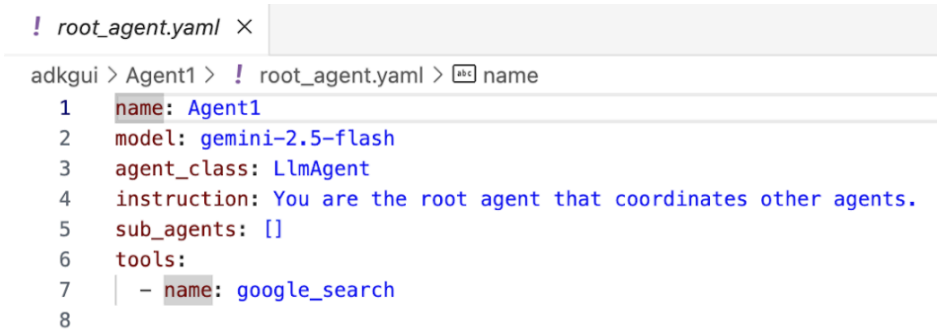


圖 15：確認 google_search 已新增為 **Agent1** 中的工具

步驟 8 · 約 0 分鐘

8. 將 Agent 部署至 Cloud Run

現在將建立的代理程式部署至 [Cloud Run](#)！透過 [Cloud Run](#)，您可以在全代管平台上迅速建構應用程式或網站。

無須管理基礎架構，即可執行前端和後端服務、批次工作、託管 LLM，以及將處理工作負載排入佇列。

在 [Cloud Shell 編輯器](#) 終端機中，如果仍在執行 [ADK \(Agent Development Kit\)](#) 伺服器，請按下 Ctrl+C 鍵停止執行。

1. 前往專案根目錄。

```
cd ~/adkui
```

2. 取得部署程式碼。執行指令後，您應該會在 [Cloud Shell 編輯器](#) 的「Explorer」窗格中看到 **deploycloudrun.py** 檔案。

```
curl -LO https://raw.githubusercontent.com/haren-  
bh/codelabs/main/adk_visual_builder/deploycloudrun.py
```

3. 請查看 `deploycloudrun.py` 中的部署選項。我們將使用 **adk deploy** 指令，將代理程式部署至 [Cloud Run](#)。[ADK \(Agent Development Kit\)](#) 內建將代理部署至 [Cloud Run](#) 的選項。我們需要指定 Google Cloud 專案 ID、區域等參數。就應用程式路徑而言，這個指令碼會假設 `agent_path=./Agent1`。我們也會建立具備必要權限的新服務帳戶，並將其附加至 [Cloud Run](#)。[Cloud Run](#) 需要存取 Vertex AI、Cloud Storage 等服務，才能執行 Agent。

```
command = [  
    "adk", "deploy", "cloud_run",  
    f"--project={project_id}",  
    f"--region={location}",  
    f"--service_name={service_name}",  
    f"--app_name={app_name}",  
    f"--artifact_service_uri=memory://",  
    f"--with_ui",  
    agent_path,  
    f"--",  
    f"--service-account={sa_email}",  
]
```

4. 執行 **deploycloudrun.py** 指令碼^{**}。部署作業應會開始，如下圖所示。^{**}

```
python3 deploycloudrun.py
```

如果收到如下所示的確認訊息，請針對所有訊息按下 Y 鍵並輸入 Enter 鍵。[deploycloudrun.py](#) 會假設代理程式位於 **Agent1** 資料夾中，如上所述。

```
per ty
❌ Creating service account: adkvisualbuilder...
📁 Assigning IAM roles to adkvisualbuilder@harentest.iam.gserviceaccount.com...
🚀 Deploying agent 'agentlapp' to harentest using adkvisualbuilder@harentest.iam.gserviceaccount.com...
WARNING: ADK Web is for development purposes. It has access to all data and should not be used in production.
Start generating Cloud Run source files in /tmp/cloud_run_deploy_src/20260107_094002
Copying agent source code...
Copying agent source code completed.
Creating Dockerfile...
Creating Dockerfile complete: /tmp/cloud_run_deploy_src/20260107_094002/Dockerfile
Deploying to Cloud Run...
The following APIs are not enabled on project [harentest]:
  artifactregistry.googleapis.com
  cloudbuild.googleapis.com
  run.googleapis.com

Do you want enable these APIs to continue (this will take a few minutes)? (Y/n)? Y
```

圖 16：將代理程式部署至 Cloud Run，並按下 Y 鍵確認所有訊息。

5. 部署完成後，您應該會看到服務網址，例如 **<https://agent1service-78833623456.us-central1.run.app>**
6. 在網路瀏覽器中存取網址，即可啟動應用程式。

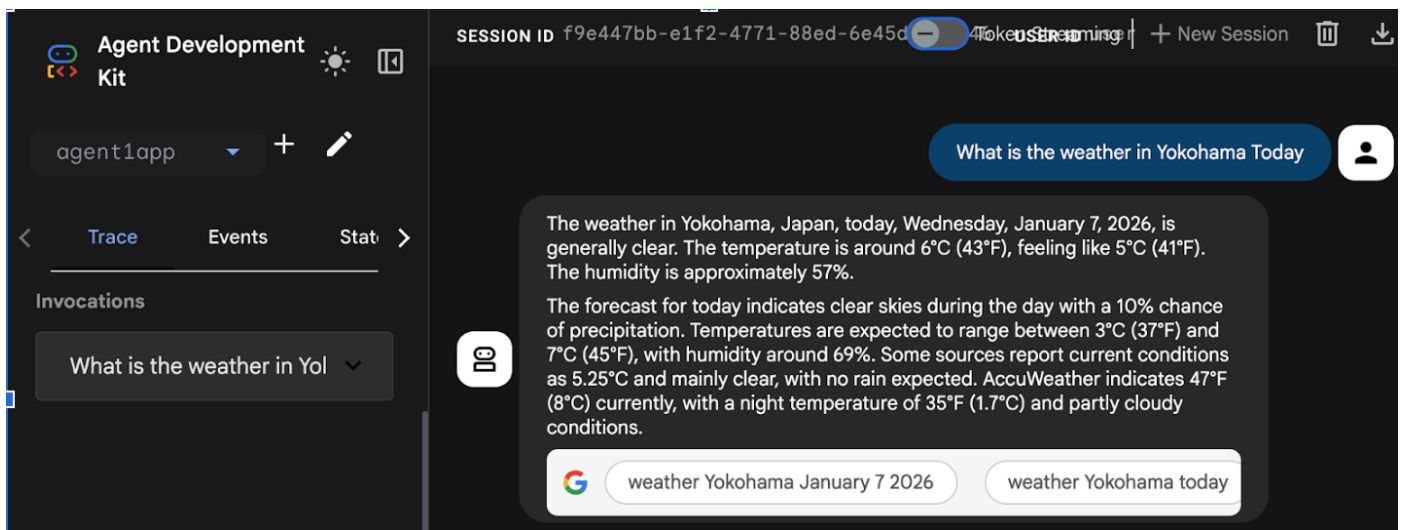


Figure 17: Agent running in Cloud Run

9. 建立具有子代理和自訂工具的代理

在上一節中，您建立了一個內建 Google 搜尋工具的單一代理程式。在本節中，您將建立多代理系統，允許代理使用自訂工具。

1. 請先重新啟動 **ADK (Agent Development Kit)** 伺服器。前往啟動 **ADK (Agent Development Kit)** 伺服器的終端機，然後按下 **CTRL+C** 鍵關閉伺服器 (如果伺服器仍在執行)。執行下列指令，再次啟動伺服器。

```
#make sure you are in the right folder.  
cd ~/adkui  
  
#start the server  
adk web
```

2. 按住 Ctrl 鍵並點選網址 (例如 <http://localhost:8000>) 顯示在畫面上。瀏覽器分頁應會顯示 **ADK (Agent Development Kit)** GUI。
3. 按一下「+」按鈕建立新的代理程式。在代理程式對話方塊中輸入「Agent2」(圖 18)，然後按一下「建立」。

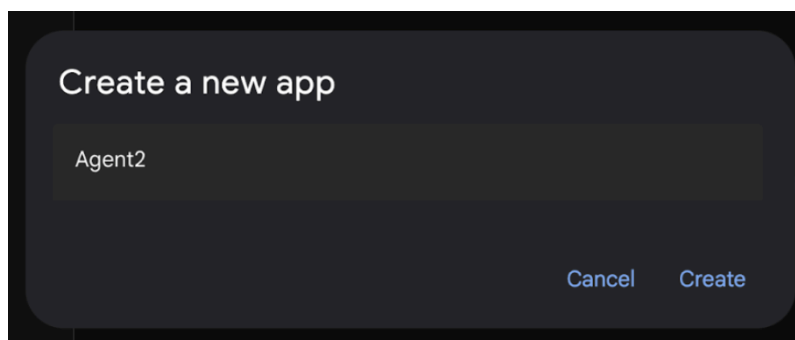


圖 18：建立新的虛擬服務專員應用程式。

4. 在 Agent2 的指令部分中，輸入下列內容。

```
You are an agent that takes image creation instruction from the user and passes it to your  
sub agent
```

5. 現在，我們要把子代理新增至根代理。如要這麼做，請按一下左窗格底部的「子代理程式」選單左側的「+」按鈕 (圖 19)，然後按一下「LLM 代理程式」。系統會建立新的代理程式，做為根代理程式的新子代理程式。

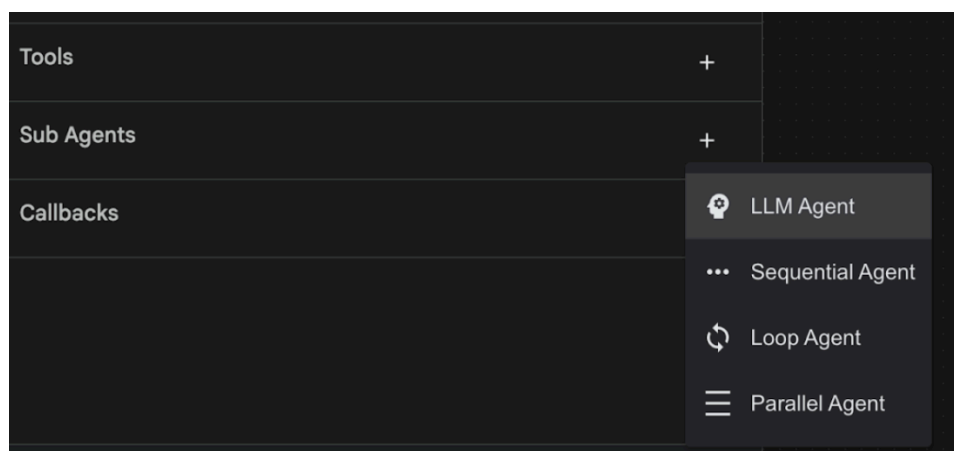


圖 19：新增子代理。

6. 在「sub_agent_1」 **sub_agent_1**的「Instructions」 (操作說明) 中輸入下列文字。

```
You are an Agent that can take instructions about an image and create an image using the  
create_image tool.
```

7. 現在，我們要在這個子代理上新增自訂工具。這項工具會呼叫 **Imagen** 模型，根據使用者的指示生成圖片。如要這麼做，請先點選上一個步驟中建立的子代理程式，然後點選「工具」選單旁的「+」按鈕。在工具選項清單中，點選「函式工具」。這項工具可讓我們在工具中加入自己的自訂程式碼。

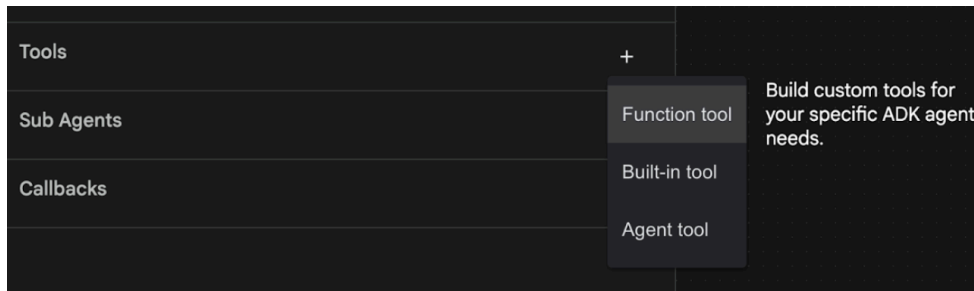


圖 20：按一下「函式」工具，建立新工具。8. 在對話方塊中，將工具命名為「Agent2.image_creation_tool.create_image」 **Agent2.image_creation_tool.create_image**。

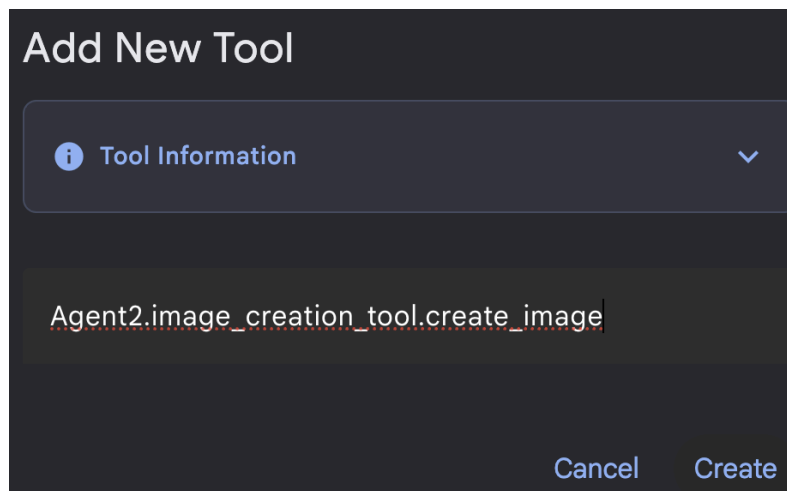


圖 21：新增工具名稱

9. 按一下「儲存」按鈕儲存變更。

10. 在 **Cloud Shell 編輯器**終端機中，按下 **Ctrl+S** 鍵關閉 **adk** 伺服器。

11. 在終端機中輸入下列指令，建立 **image_creation_tool.py** 檔案。

```
touch ~/adkui/Agent2/image_creation_tool.py
```

12. 在 **Cloud Shell 編輯器**的「Explorer」窗格中，按一下新建立的 **image_creation_tool.py** 開啟該檔案。將 **image_creation_tool.py** 的內容替換為下列內容，然後儲存 (**Ctrl+S**)。

```

import os
import io
import vertexai
from vertexai.preview.vision_models import ImageGenerationModel
from dotenv import load_dotenv
import uuid
from typing import Union
from datetime import datetime
from google import genai
from google.adk.tools import ToolContext
import logging

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

async def create_image(prompt: str, tool_context: ToolContext) -> Union[bytes, str]:
    """
    Generates an image based on a text prompt using a Vertex AI Imagen model.
    Args:
        prompt: The text prompt to generate the image from.

    Returns:
        The binary image data (PNG format) on success, or an error message string on failure.
    """
    print(f"Attempting to generate image for prompt: '{prompt}'")

    try:
        # Load environment variables from .env file two levels up
        dotenv_path = os.path.join(os.path.dirname(__file__), '..', '..', '.env')
        load_dotenv(dotenv_path=dotenv_path)
        project_id = os.getenv("GOOGLE_CLOUD_PROJECT")
        location = os.getenv("GOOGLE_CLOUD_LOCATION")
        model_name = os.getenv("IMAGEN_MODEL")
        client = genai.Client(
            vertexai=True,
            project=project_id,
            location=location,
        )
        response = client.models.generate_images(
            model="imagen-3.0-generate-002",
            prompt=prompt,
            config=types.GenerateImagesConfig(
                number_of_images=1,
                aspect_ratio="9:16",
                safety_filter_level="block_low_and_above",
                person_generation="allow_adult",
            ),
        )
        if not all([project_id, location, model_name]):
            return "Error: Missing GOOGLE_CLOUD_PROJECT, GOOGLE_CLOUD_LOCATION, or IMAGEN_MODEL"
    
```

```

in .env file."
    vertexai.init(project=project_id, location=location)
    model = ImageGenerationModel.from_pretrained(model_name)
    images = model.generate_images(
        prompt=prompt,
        number_of_images=1
    )
    if response.generated_images is None:
        return "Error: No image was generated."
    for generated_image in response.generated_images:
        # Get the image bytes
        image_bytes = generated_image.image.image_bytes
        counter = str(tool_context.state.get("loop_iteration", 0))
        artifact_name = f"generated_image_" + counter + ".png"
        # Save as ADK artifact (optional, if still needed by other ADK components)
        report_artifact = types.Part.from_bytes(
            data=image_bytes, mime_type="image/png"
        )
        await tool_context.save_artifact(artifact_name, report_artifact)
        logger.info(f"Image also saved as ADK artifact: {artifact_name}")
        return {
            "status": "success",
            "message": f"Image generated . ADK artifact: {artifact_name}.",
            "artifact_name": artifact_name,
        }
    except Exception as e:
        error_message = f"An error occurred during image generation: {e}"
        print(error_message)
        return error_message

```

13. 請先重新啟動 [ADK \(Agent Development Kit\)](#) 伺服器。前往啟動 [ADK \(Agent Development Kit\)](#) 伺服器的終端機，然後按下 **CTRL+C** 鍵關閉伺服器 (如果伺服器仍在執行)。執行下列指令，再次啟動伺服器。

```

#make sure you are in the right folder.
cd ~/adkui

#start the server
adk web

```

14. 按住 Ctrl 鍵並點選網址 (例如 <http://localhost:8000>) 顯示在畫面上。瀏覽器分頁應會顯示 [ADK \(Agent Development Kit\)](#) GUI。

Note:

In case you get an error saying port 8000 is already in use, use the following command to kill the service and restart it (Run step 14).

```
ss -lntcp | grep ":8000" | awk '{print $NF}' | grep -oP 'pid=\K\d+' | xargs -r kill -9
```

15. 在「ADK (Agent Development Kit)」使用者介面分頁中，選取「Agent list」中的「Agent2」，然後按下編輯按鈕 (筆圖示)。在 [ADK \(Agent Development Kit\)](#) 視覺化編輯器中，按一下「儲存」按鈕保留變更。
16. 現在可以測試新的代理程式。
17. 在 [ADK \(Agent Development Kit\)](#) UI 對話介面中輸入下列提示。你也可以試試其他提示。您應該會看到圖 22 所示的結果。

Create an image of a cat

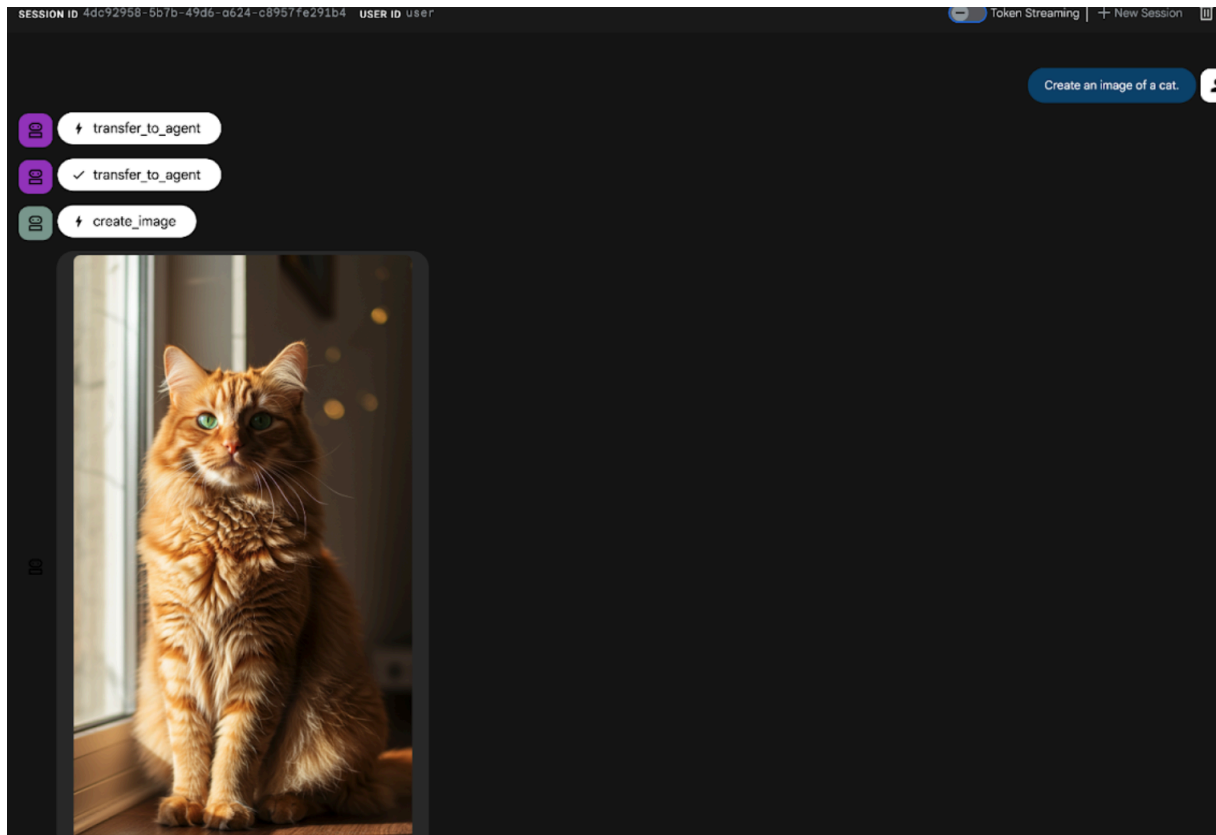


圖 22 : ADK UI 對話介面

10. 建立工作流程代理程式

上一個步驟是建構具備子代理和專業圖像創作工具的代理，這個階段的重點則是提升代理的功能。我們會先確保使用者的初始提示經過最佳化，再生成圖片，藉此提升流程。為此，系統會將 Sequential 代理程式整合至 Root 代理程式，以處理下列兩步驟工作流程：

1. 接收來自根代理程式的提示，並進行提示強化。
2. 將修正後的提示詞轉送給圖片建立代理程式，使用 IMAGEN 生成最終圖片。
3. 請先重新啟動 **ADK (Agent Development Kit)** 伺服器。前往啟動 **ADK (Agent Development Kit)** 伺服器的終端機，然後按下 **CTRL+C** 鍵關閉伺服器 (如果伺服器仍在執行)。執行下列指令，再次啟動伺服器。

```
#make sure you are in the right folder.
cd ~/adkui

#start the server
adk web
```

2. 按住 Ctrl 鍵並點選網址 (例如 <http://localhost:8000>) 顯示在畫面上。瀏覽器分頁應會顯示 **ADK (Agent Development Kit)** GUI。
3. 從代理程式選取器選取「Agent2」，然後按一下「編輯」按鈕 (筆圖示)。
4. 按一下「Agent2 (Root Agent)」，然後按一下「Sub Agents」(子代理程式) 選單旁邊的「+」按鈕。然後從選項清單中按一下「Sequential Agent」
5. 您應該會看到類似圖 23 的代理程式結構

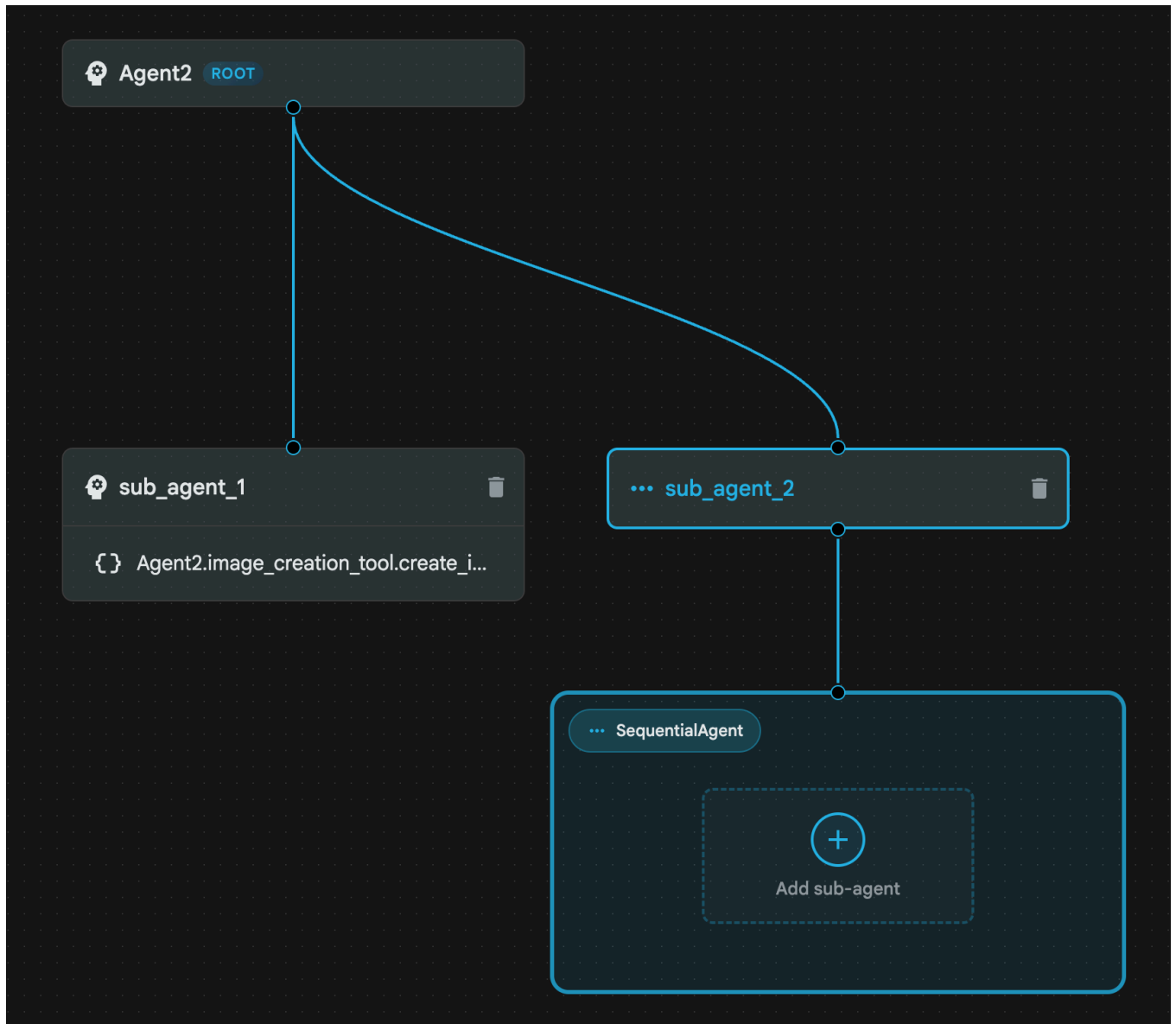


圖 23：循序代理程式結構

- 現在我們要將第一個代理新增至 **Sequential Agent**，做為提示強化工具。如要這麼做，請按一下 SequentialAgent 方塊中的「新增子代理程式」按鈕，然後按一下「LLM 代理程式」。
- 我們需要在序列中新增另一個代理程式，因此請重複步驟 6，新增另一個 LLM 代理程式 (按下「+」按鈕，然後選取「LLMAgent」)。
- 按一下 **sub_agent_4**，然後按一下左側窗格「工具」旁的「+」圖示，新增工具。從選項中點選「函數工具」。在「命名工具」對話方塊中，輸入 **Agent2.image_creation_tool.create_image**，然後按下「建立」。
- 現在可以刪除 **sub_agent_1**，因為更進階的 **sub_agent_2** 已取代前者。如要刪除，請按一下圖表中「sub_agent_1」右側的「刪除」按鈕。

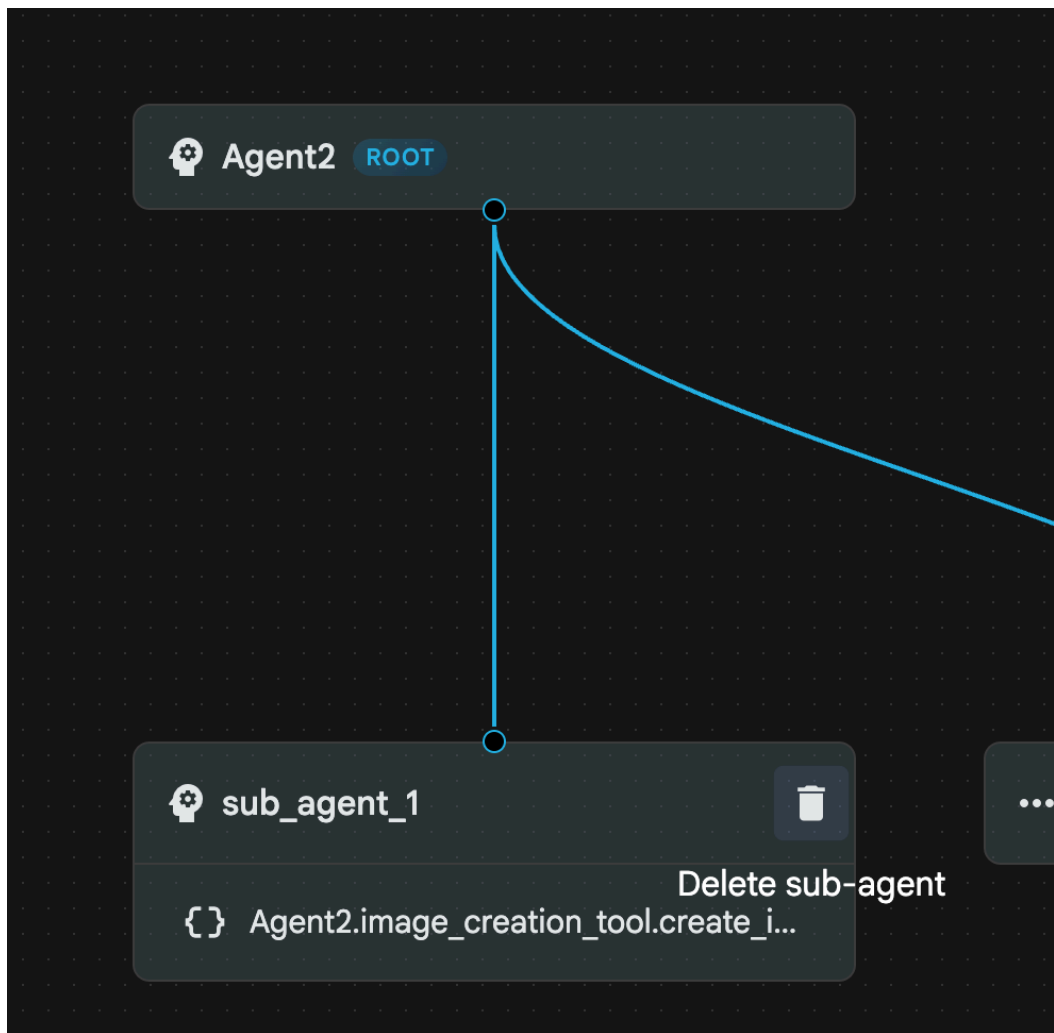


圖 24：刪除 sub_agent_1 10。我們的代理程式結構如圖 25 所示。

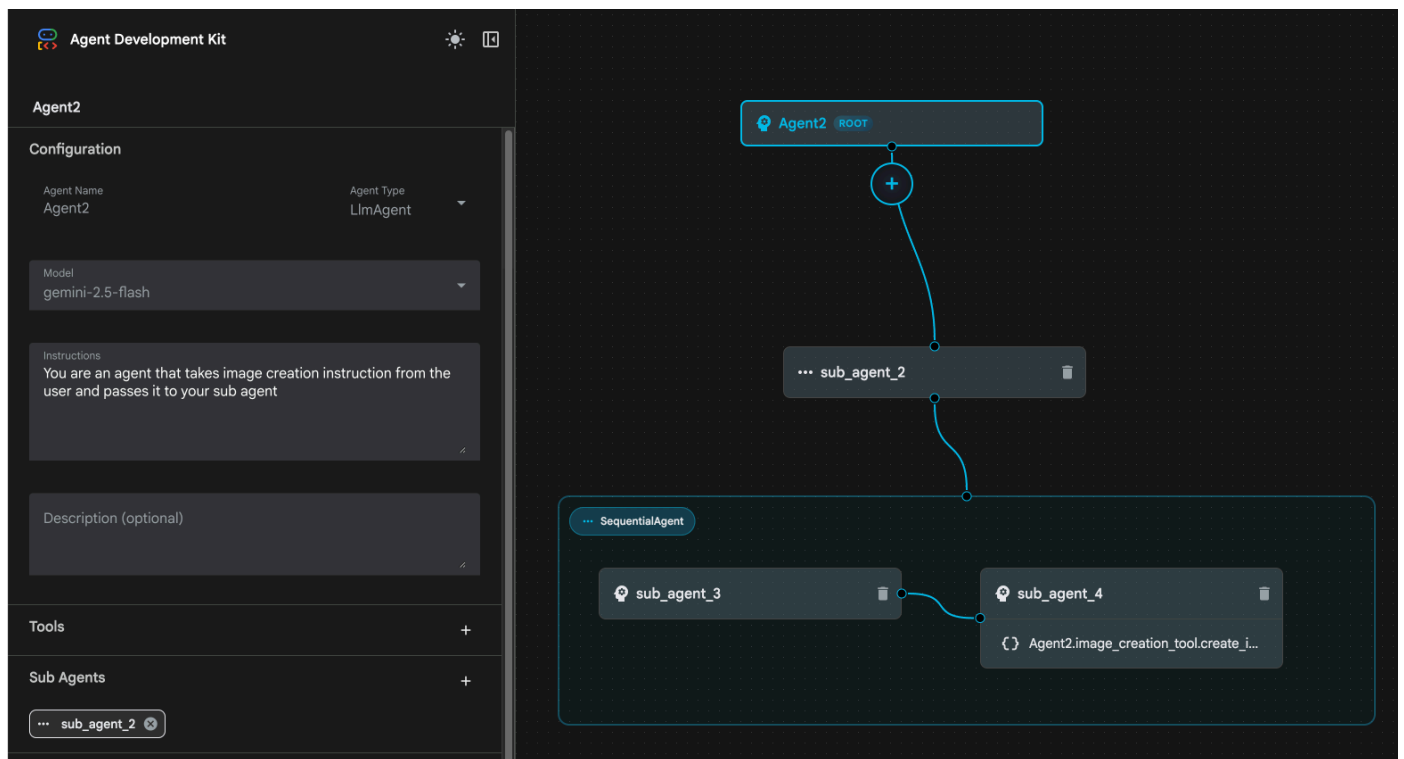


圖 25：強化型代理程式的最終結構

11. 按一下 sub_agent_3，然後在指令中輸入下列內容。

Act as a professional AI Image Prompt Engineer. I will provide you with a basic idea for an image. Your job is to expand my idea into a detailed, high-quality prompt for models like Imagen.

For every input, output the following structure:

1. ****Optimized Prompt****: A vivid, descriptive paragraph including subject, background, lighting, and textures.
2. ****Style & Medium****: Specify if it is photorealistic, digital art, oil painting, etc.
3. ****Camera & Lighting****: Define the lens (e.g., 85mm), angle, and light quality (e.g., volumetric, golden hour).

Guidelines: Use sensory language, avoid buzzwords like 'photorealistic' unless necessary, and focus on specific artistic descriptors.

Once the prompt is created send the prompt to the

12. 點選「sub_agent_4」 **sub_agent_4**。將指令變更為下列內容。

You are an agent that takes instructions about an image and can generate the image using the create_image tool.

13. 按一下「儲存」按鈕

14. 前往 Cloud Shell 編輯器 Explorer 窗格，然後開啟代理程式 YAML 檔案。代理程式檔案應如下所示

root_agent.yaml

```
name: Agent2
model: gemini-2.5-flash
agent_class: LlmAgent
instruction: You are an agent that takes image creation instruction from the
  user and passes it to your sub agent
sub_agents:
  - config_path: ./sub_agent_2.yaml
tools: []
```

sub_agent_2.yaml

```
name: sub_agent_2
agent_class: SequentialAgent
sub_agents:
  - config_path: ./sub_agent_3.yaml
  - config_path: ./sub_agent_4.yaml
```

```
sub_agent_3.yaml

name: sub_agent_3
model: gemini-2.5-flash
agent_class: LlmAgent
instruction: |
  Act as a professional AI Image Prompt Engineer. I will provide you with a
  basic idea for an image. Your job is to expand my idea into a detailed,
  high-quality prompt for models like Imagen.

  For every input, output the following structure: 1. Optimized Prompt: A
  vivid, descriptive paragraph including subject, background, lighting, and
  textures. 2. Style & Medium: Specify if it is photorealistic, digital
  art, oil painting, etc. 3. Camera & Lighting: Define the lens (e.g.,
  85mm), angle, and light quality (e.g., volumetric, golden hour).

  Guidelines: Use sensory language, avoid buzzwords like
  'photorealistic' unless necessary, and focus on specific artistic
  descriptors. Once the prompt is created send the prompt to the
sub_agents: []
tools: []
```

```
sub_agent_4.yaml

name: sub_agent_4
model: gemini-2.5-flash
agent_class: LlmAgent
instruction: You are an agent that takes instructions about an image and
  generate the image using the create_image tool.
sub_agents: []
tools:
  - name: Agent2.image_creation_tool.create_image
```

15. 現在來測試看看。

16. 請先重新啟動 [ADK \(Agent Development Kit\)](#) 伺服器。前往啟動 [ADK \(Agent Development Kit\)](#) 伺服器的終端機，然後按下 **CTRL+C** 鍵關閉伺服器 (如果伺服器仍在執行)。執行下列指令，再次啟動伺服器。

```
#make sure you are in the right folder.
cd ~/adkui

#start the server
adk web
```

17. 按住 Ctrl 鍵並點選網址 (例如 <http://localhost:8000>) 顯示在畫面上。瀏覽器分頁應會顯示 [ADK \(Agent Development Kit\)](#) GUI。

18. 從服務專員清單中選取「服務專員 2」。然後輸入下列提示詞。

```
Create an image of a Cat
```

19. 在 Agent 運作期間，您可以查看 [Cloud Shell 編輯器](#) 中的終端機，瞭解背景作業的進度。最終結果應如圖 26 所示。

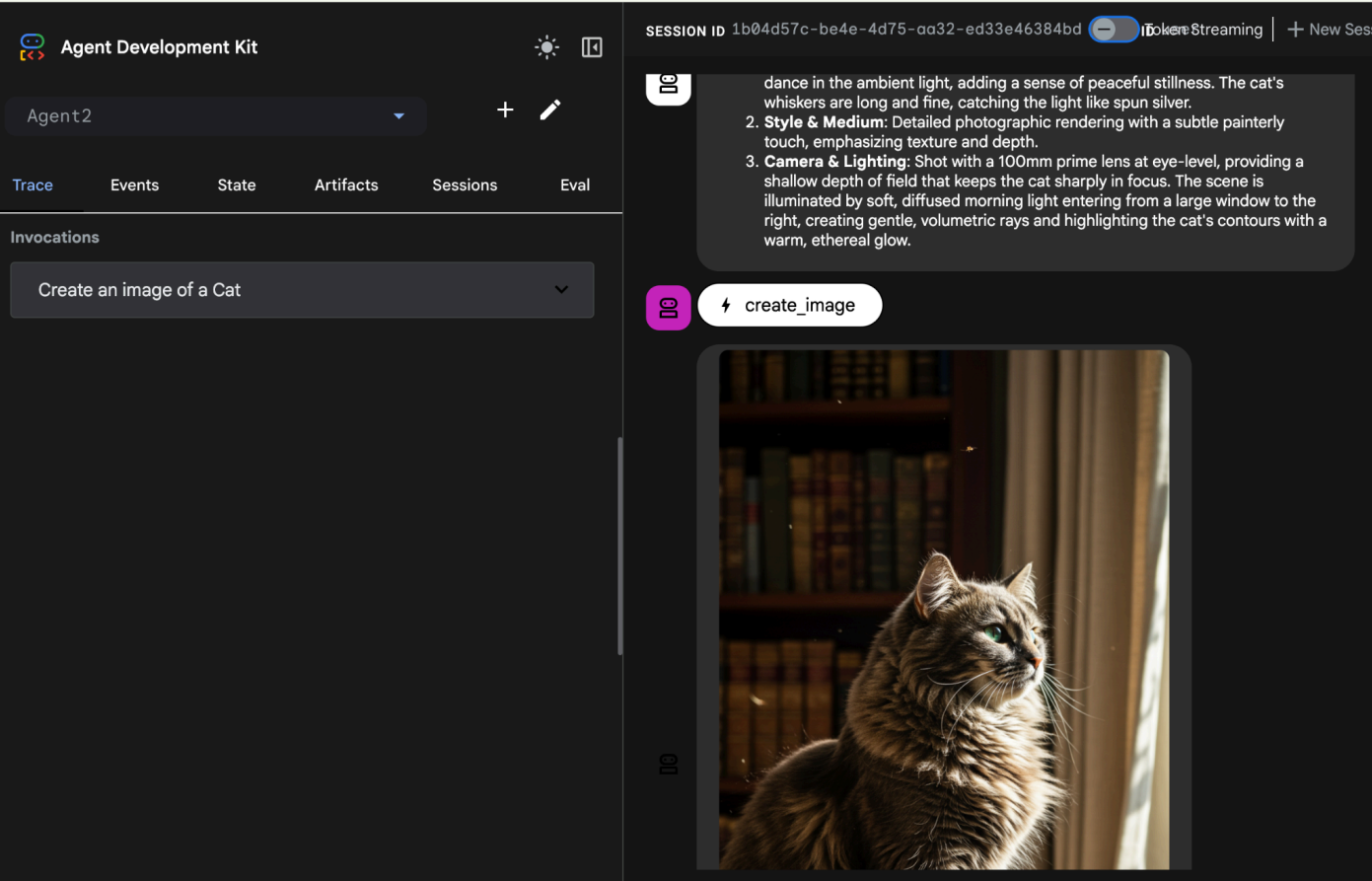


圖 26：測試代理

11. 使用 Agent Builder 助理建立代理

Agent Builder Assistant 是 [ADK Visual Builder](#) 的一部分，可透過簡單的聊天介面中的提示，以互動方式建立代理，並支援不同程度的複雜度。使用 [ADK Visual Builder](#)，即可立即取得所開發代理程式的視覺回饋。在本實驗室中，我們將建構一個代理，能夠根據使用者的要求生成 HTML 格式的漫畫書。使用者可以提供簡單的提示，例如「製作漢賽爾與葛麗特的故事漫畫」，也可以輸入完整的故事。接著，代理程式會分析敘事內容、將其劃分為多個面板，並使用 Nanobanana 製作漫畫圖像，最終將結果封裝為 HTML 格式。

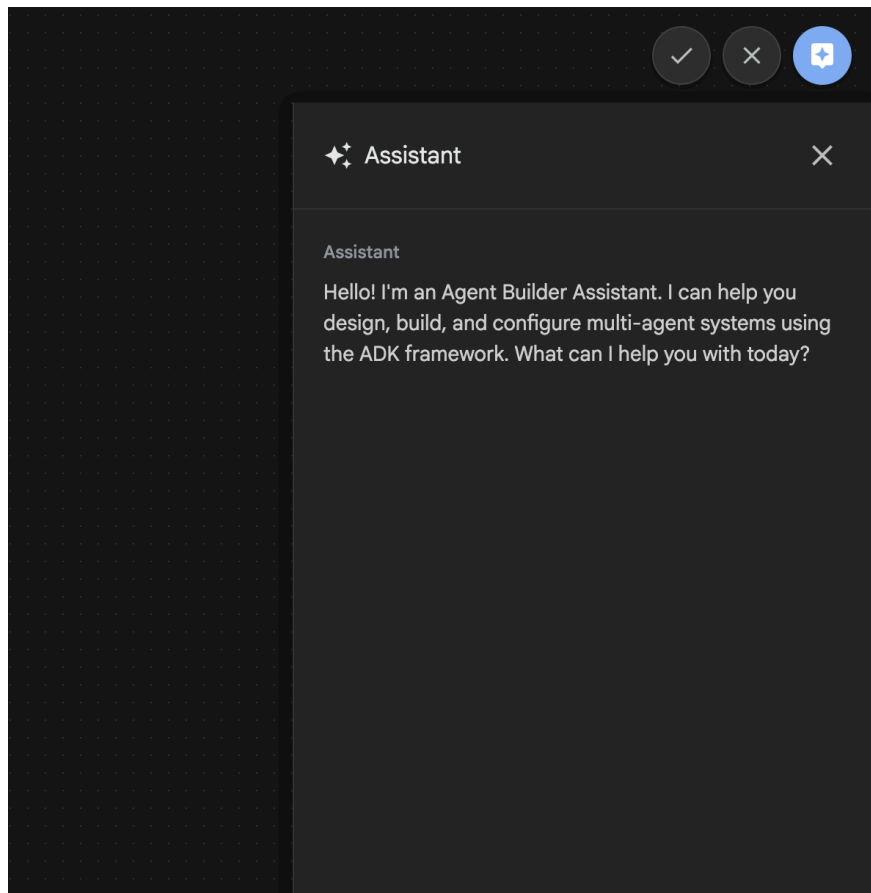


圖 27：Agent Builder 助理 UI

立即開始！

注意：

請注意，使用 Agent Builder Assistant 建立新代理程式時，由於 LLM 的輸出內容可能有所不同，因此按照下列指示建立的代理程式可能與範例不盡相同。請按照這些步驟建立代理程式。

1. 請先重新啟動 [ADK \(Agent Development Kit\)](#) 伺服器。前往啟動 [ADK \(Agent Development Kit\)](#) 伺服器的終端機，然後按下 **CTRL+C** 鍵關閉伺服器 (如果伺服器仍在執行)。執行下列指令，再次啟動伺服器。

```
#make sure you are in the right folder.
cd ~/adkui

#start the server
adk web
```

2. 按住 Ctrl 鍵並點選網址 (例如 <http://localhost:8000>) 顯示在畫面上。瀏覽器分頁應會顯示 [ADK \(Agent Development Kit\)](#) GUI。
3. 在 [ADK \(Agent Development Kit\)](#) GUI 中，按一下「+」按鈕，建立新的代理程式。
4. 在對話方塊中輸入「Agent3」，然後按一下「建立」按鈕。

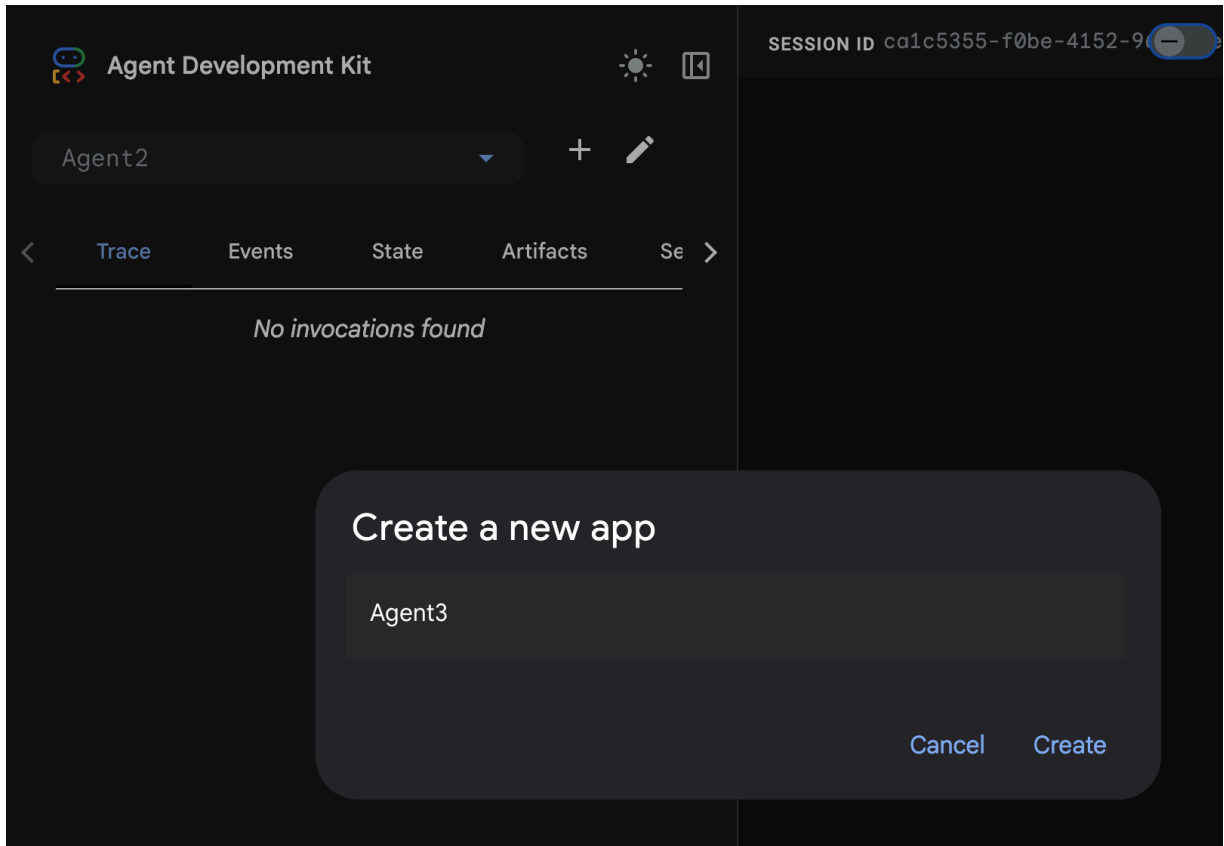


圖 28：建立新的代理程式 Agent3

5. 在右側的「助理」窗格中輸入下列提示。下方的提示包含建立代理系統的所有必要指令，可建立以 HTML 為基礎的代理。

System Goal: You are the Studio Director (Root Agent). Your objective is to manage a linear pipeline of four ADK Sequential Agents to transform a user's seed idea into a fully rendered, responsive HTML5 comic book.

0. Root Agent: The Studio Director

Role: Orchestrator and State Manager.

Logic: Receives the user's initial request. It initializes the workflow and ensures the output of each Sub-Agent is passed as the context for the next. It monitors the sequence to ensure no steps are skipped. Make sure the query explicitly mentions "Create me a comic of ..." if it's just a general question or prompt just answer the question.

1. Sub-Agent: The Scripting Agent (Sequential Step 1)

Role: Narrative & Character Architect.

Input: Seed idea from Root Agent.

Logic:

1. Create a Character Manifest: Define 3 specific, unchangeable visual identifiers for every character (e.g., "Gretel: Blue neon hair ribbons, silver apron, glowing boots").
2. Expand the seed idea into a coherent narrative arc.

Output: A narrative script and a mandatory character visual guide.

2. Sub-Agent: The Panelization Agent (Sequential Step 2)

Role: Cinematographer & Storyboarder.

Input: Script and Character Manifest from Step 1.

Logic:

1. Divide the script into exactly X panels (User-defined or default to 8).
2. For each panel, define a specific composition (e.g., "Panel 1: Wide shot of the gingerbread house").

Output: A structured list of exactly X panel descriptions.

3. Sub-Agent: The Image Synthesis Agent (Sequential Step 3)

Role: Technical Artist & Asset Generator.

Input: The structured list of panel descriptions from Step 2.

Logic:

1. Iterative Generation: You must execute the "generate_image" tool in "image_generation.py" file (Nano Banana) individually for each panel defined in Step 2.
2. Prompt Engineering: For every panel, translate the description into a Nano Banana prompt, strictly enforcing the character identifiers (e.g., the "blue neon ribbons") and the global style: "vibrant comic book style, heavy ink lines, cel-shaded, 4k." . Make sure that the necessary speech bubbles are present in the image representing the dialogue.
3. Mapping: Associate each generated image URL with its corresponding panel

number and dialogue.

Output: A complete gallery of X images mapped to their respective panel data.

4. Sub-Agent: The Assembly Agent (Sequential Step 4)

Role: Frontend Developer.

Input: The mapped images and panel text from Step 3.

Logic:

1. Write a clean, responsive HTML5/CSS3 file that shows the comic. The comic should be Scrollable with image on the top and the description below the image.
2. Use "write_comic_html" tool in file_writer.py to write the created html file in the "output" folder.
4. In the "write_comic_html" tool add logic to copy the images folder to the output folder so that the images in the html file are actually visible when the user opens the html file.

Output: A final, production-ready HTML code block.

6. 代理程式可能會要求您輸入要使用的模型，請從提供的選項中輸入 **gemini-2.5-pro**。

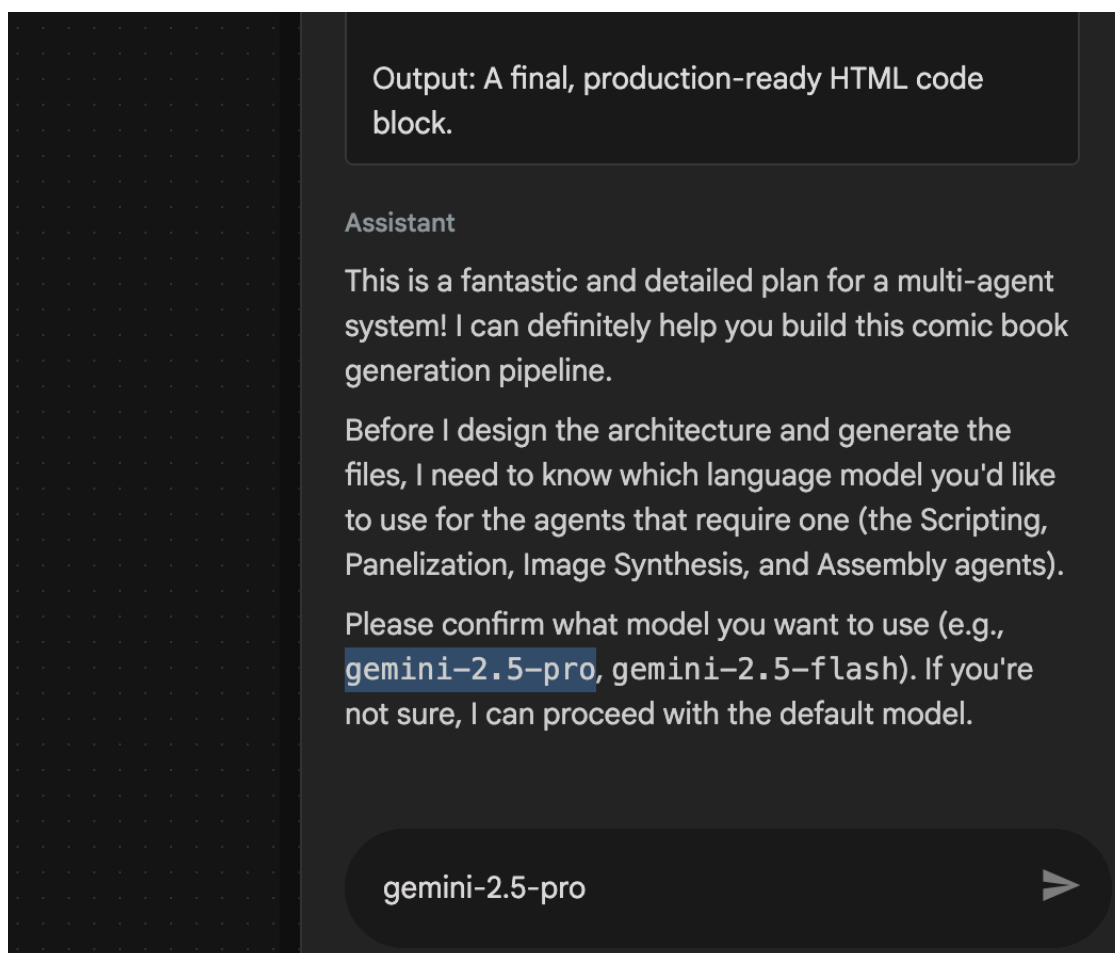


圖 29：如果系統提示輸入要使用的模型，請輸入 gemini-2.5-pro

7. Google 助理可能會提供行程，並詢問是否要繼續。檢查計畫，然後輸入「OK」並按下「Enter」。

```
files would exist. Here, we'll create
# placeholder files to simulate the
copy.
    for image_url in image_urls:
        if
image_url.startswith("images/"):
            # Create a dummy source file
to simulate its existence
            dummy_source_path =
os.path.basename(image_url)
            if not
os.path.exists(dummy_source_path):
                with
open(dummy_source_path, "w") as dummy_f:

dummy_f.write("placeholder image")

            # Copy the dummy file to the
output images directory

shutil.copy(dummy_source_path,
images_dir)
            # Clean up the dummy source
file
            os.remove(dummy_source_path)

    return f"Successfully created comic
book. You can find it at:
{html_file_path}"

Should I proceed with creating these files?
```

OK



圖 30：如果方案沒問題，請輸入「OK」8。Google 助理完成作業後，您應該會看到代理程式結構，如圖 31 所示。

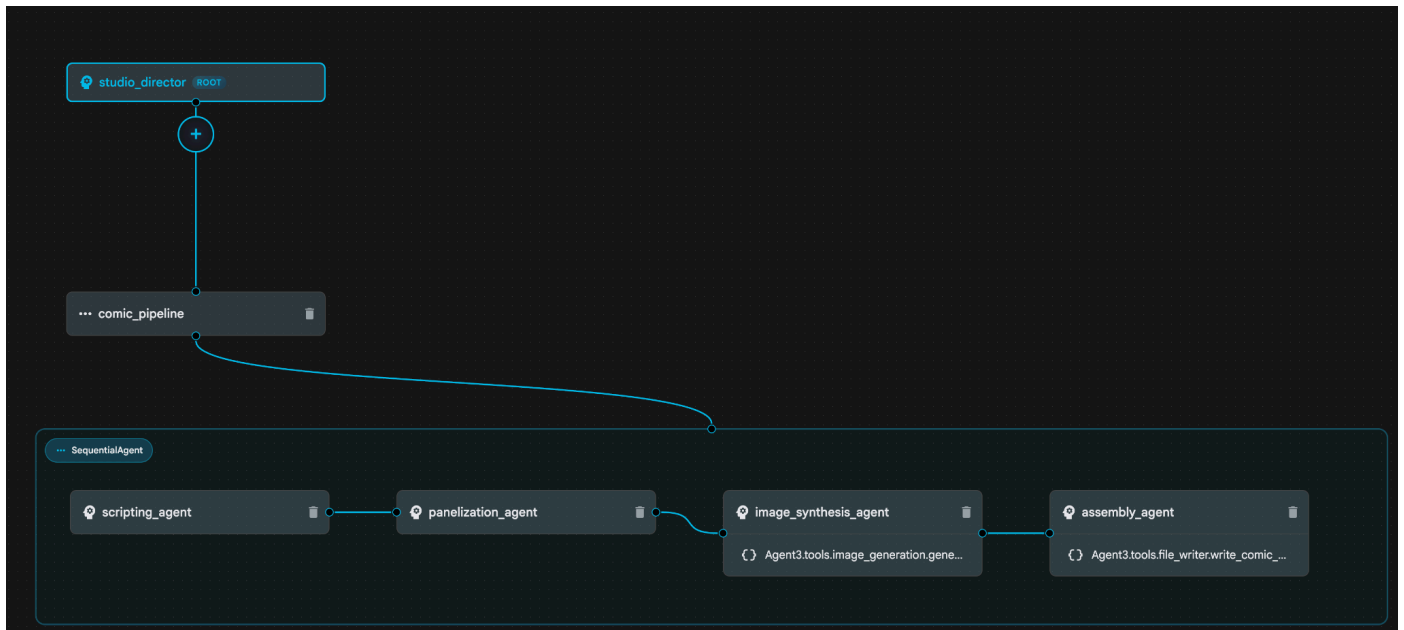


圖 31：Agent Builder Assistant 9 建立的代理程式。在 `image_synthesis_agent` (您的名稱可能不同) 內，按一下「`Agent3.tools.image_generation.gene...`」工具。如果工具名稱的最後一個部分不是 **`image_generation.generate_image`**，請變更為 **`image_generation.generate_image`**。如果名稱已設為該名稱，則無須變更。按下「儲存」按鈕即可儲存。

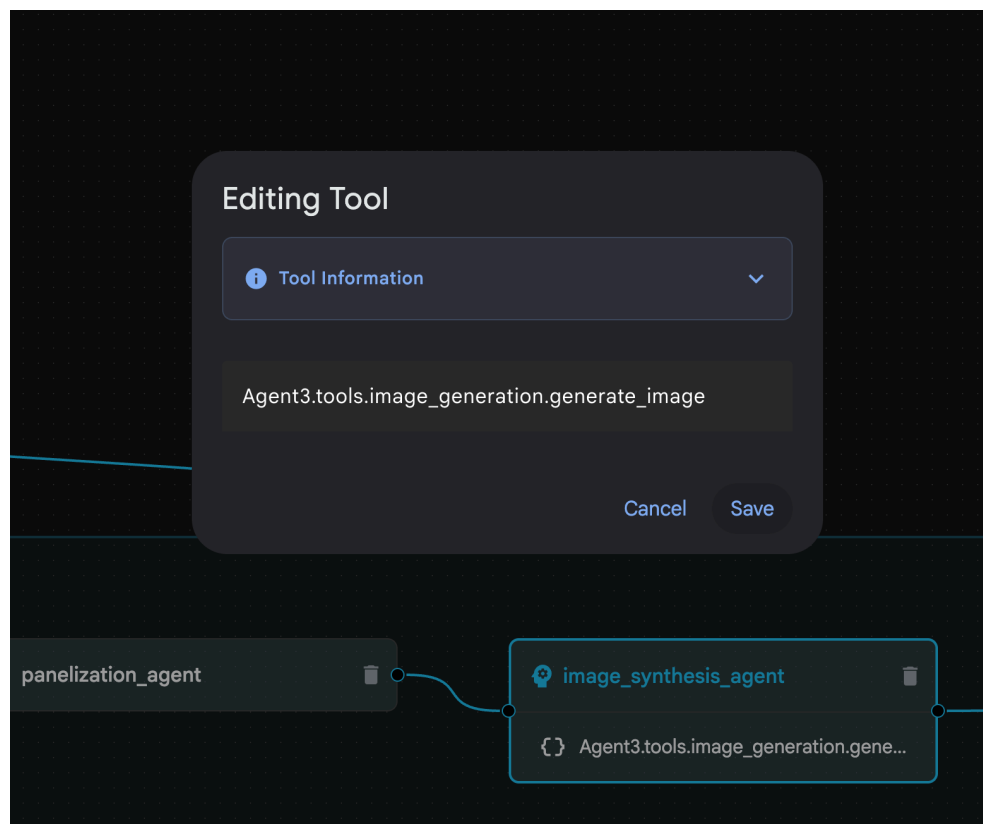


圖 32：將工具名稱變更為 `image_generation.generate_image`，然後按一下「儲存」。

10. 在 `assembly_agent` (您的代理程式名稱可能不同) 內，按一下 **`**Agent3.tools.file_writer.write_comic_...**`** 工具。如果工具名稱的最後一個部分不是 **`**file_writer.write_comic_html**`**，請將其變更為 **`**file_writer.write_comic_html**`**。

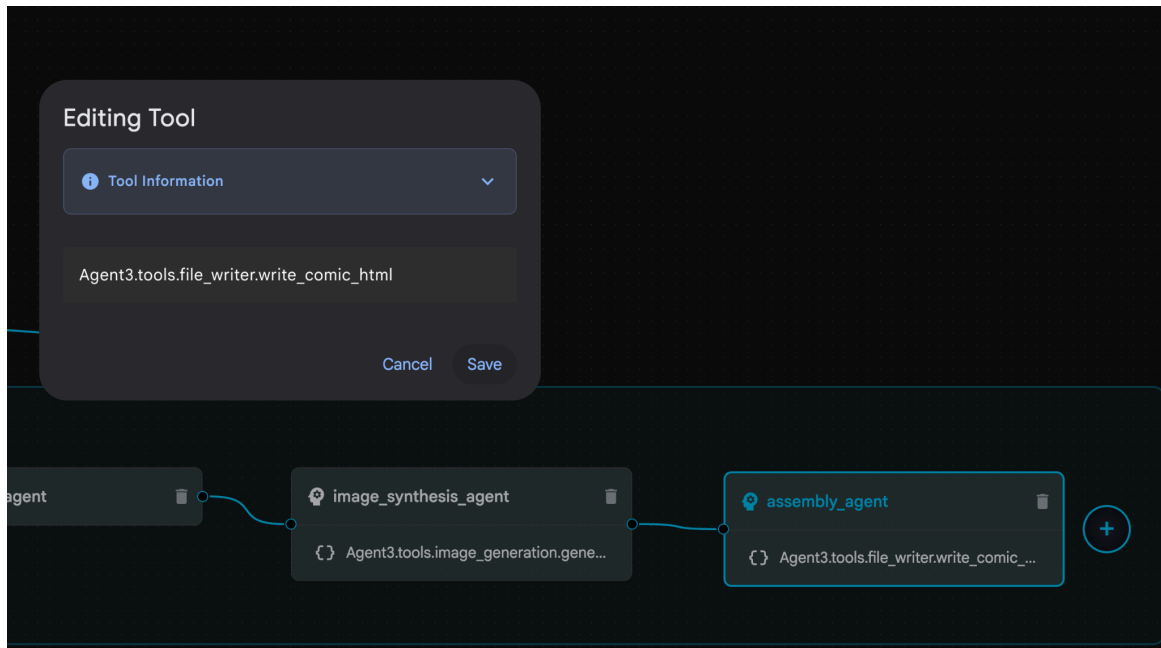


圖 33：將工具名稱變更為 **file_writer.write_comic_html** 11. 按下左側面板左下方的「儲存」按鈕，儲存新建立的代理程式。12. 在 [Cloud Shell 編輯器](#) 的「Explorer」窗格中，展開 **Agent3** 資料夾，**Agent3/** 資料夾內應有 **tools** 資料夾。點選「Agent3/tools/file_writer.py」開啟檔案，然後將「Agent3/tools/file_writer.py」的內容替換為下列程式碼。按下 **Ctrl+S** 鍵儲存。注意：雖然 **Agent Assist** 可能已建立正確的程式碼，但本實驗室會使用經過測試的程式碼。

```

import os
import shutil

def write_comic_html(html_content: str, image_directory: str = "images") -> str:
    """
    Writes the final HTML content to a file and copies the image assets.

    Args:
        html_content: A string containing the full HTML of the comic.
        image_directory: The source directory where generated images are stored.

    Returns:
        A confirmation message indicating success or failure.
    """
    output_dir = "output"
    images_output_dir = os.path.join(output_dir, image_directory)

    try:
        # Create the main output directory
        if not os.path.exists(output_dir):
            os.makedirs(output_dir)

        # Copy the entire image directory to the output folder
        if os.path.exists(image_directory):
            if os.path.exists(images_output_dir):
                shutil.rmtree(images_output_dir) # Remove old images
            shutil.copytree(image_directory, images_output_dir)
        else:
            return f"Error: Image directory '{image_directory}' not found."

        # Write the HTML file
        html_file_path = os.path.join(output_dir, "comic.html")
        with open(html_file_path, "w") as f:
            f.write(html_content)

        return f"Successfully created comic at '{html_file_path}'"

    except Exception as e:
        return f"An error occurred: {e}"

```

- 在 [Cloud Shell 編輯器](#) 的「Explorer」窗格中，展開 **Agent3** 資料夾，**Agent3/**資料夾內應有 **tools** 資料夾。按一下「Agent3/tools/image_generation.py」開啟檔案，然後將「Agent3/tools/image_generation.py」的內容替換為下列程式碼。按下 Ctrl+S 鍵即可儲存。注意：雖然 **Agent Assist** 可能已建立正確的程式碼，但我們會在本次實驗室中使用經過測試的程式碼。

```

import time
import os
import io
import vertexai
from vertexai.preview.vision_models import ImageGenerationModel
from dotenv import load_dotenv
import uuid
from typing import Union
from datetime import datetime
from google import genai
from google.genai import types
from google.adk.tools import ToolContext

import logging
import asyncio

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# It's better to initialize the client once and reuse it.
# IMPORTANT: Your Google Cloud Project ID must be set as an environment variable
# for the client to authenticate correctly.

def edit_image(client, prompt: str, previous_image: str, model_id: str) -> Union[bytes,
None]:
    """
    Calls the model to edit an image based on a prompt.

    Args:
        prompt: The text prompt for image editing.
        previous_image: The path to the image to be edited.
        model_id: The model to use for the edit.

    Returns:
        The raw image data as bytes, or None if an error occurred.
    """

    try:
        with open(previous_image, "rb") as f:
            image_bytes = f.read()

        response = client.models.generate_content(
            model=model_id,
            contents=[
                types.Part.from_bytes(
                    data=image_bytes,
                    mime_type="image/png", # Assuming PNG, adjust if necessary
                ),
                prompt,
            ],
        ),
    
```

```

        config=types.GenerateContentConfig(
            response_modalities=['IMAGE'],
        )
    )

    # Extract image data
    for part in response.candidates[0].content.parts:
        if part.inline_data:
            return part.inline_data.data

    logger.warning("Warning: No image data was generated for the edit.")
    return None

except FileNotFoundError:
    logger.error(f"Error: The file {previous_image} was not found.")
    return None
except Exception as e:
    logger.error(f"An error occurred during image editing: {e}")
    return None

async def generate_image(tool_context: ToolContext, prompt: str, image_name: str,
previous_image: str = None) -> dict:
    """
    Generates or edits an image and saves it to the 'images/' directory.

    If 'previous_image' is provided, it edits that image. Otherwise, it generates a new one.

    Args:
        prompt: The text prompt for the operation.
        image_name: The desired name for the output image file (without extension).
        previous_image: Optional path to an image to be edited.

    Returns:
        A confirmation message with the path to the saved image or an error message.
    """
    load_dotenv()
    project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
    if not project_id:
        return "Error: GOOGLE_CLOUD_PROJECT environment variable is not set."

    try:
        client = genai.Client(vertexai=True, project=project_id, location="global")
    except Exception as e:
        return f"Error: Failed to initialize genai.Client: {e}"

    image_data = None
    model_id = "gemini-3-pro-image-preview"

    try:
        if previous_image:
            logger.info(f"Editing image: {previous_image}")
            image_data = edit_image(

```



```

        client=client,
        prompt=prompt,
        previous_image=previous_image,
        model_id=model_id
    )
else:
    logger.info("Generating new image")
    # Generate the image
    response = client.models.generate_content(
        model=model_id,
        contents=prompt,
        config=types.GenerateContentConfig(
            response_modalities=['IMAGE'],
            image_config=types.ImageConfig(aspect_ratio="16:9"),
        ),
    )

    # Check for errors
    if response.candidates[0].finish_reason != types.FinishReason.STOP:
        return f"Error: Image generation failed. Reason: {response.candidates[0].finish_reason}"

    # Extract image data
    for part in response.candidates[0].content.parts:
        if part.inline_data:
            image_data = part.inline_data.data
            break

    if not image_data:
        return {"status": "error", "message": "No image data was generated.",
            "artifact_name": None}

    # Create the images directory if it doesn't exist
    output_dir = "images"
    os.makedirs(output_dir, exist_ok=True)

    # Save the image to file system
    file_path = os.path.join(output_dir, f"{image_name}.png")
    with open(file_path, "wb") as f:
        f.write(image_data)

    # Save as ADK artifact
    counter = str(tool_context.state.get("loop_iteration", 0))
    artifact_name = f"{image_name}_{counter}.png"
    report_artifact = types.Part.from_bytes(data=image_data, mime_type="image/png")
    await tool_context.save_artifact(artifact_name, report_artifact)
    logger.info(f"Image also saved as ADK artifact: {artifact_name}")

    return {
        "status": "success",
        "message": f"Image generated and saved to {file_path}. ADK artifact: {artifact_name}.",
    }

```

```

        "artifact_name": artifact_name,
    }

except Exception as e:
    return f"An error occurred: {e}"

```

14. 以下提供作者環境中產生的最終 YAML 檔案供您參考 (請注意，您環境的檔案可能略有不同)。請確認代理程式 YAML 結構與 [ADK 視覺化建構工具](#) 中顯示的版面配置相符。

```

root_agent.yamlname: studio_director
model: gemini-2.5-pro
agent_class: LlmAgent
description: The Studio Director who manages the comic creation pipeline.
instruction: >
  You are the Studio Director. Your objective is to manage a linear pipeline of
  four sequential agents to transform a user's seed idea into a fully rendered,
  responsive HTML5 comic book.

  Your role is to be the primary orchestrator and state manager. You will
  receive the user's initial request.

  **Workflow:**

  1. If the user's prompt starts with "Create me a comic of ...", you must
  delegate the task to your sub-agent to begin the comic creation pipeline.

  2. If the user asks a general question or provides a prompt that does not
  explicitly ask to create a comic, you must answer the question directly
  without triggering the comic creation pipeline.

  3. Monitor the sequence to ensure no steps are skipped. Ensure the output of
  each Sub-Agent is passed as the context for the next.
sub_agents:
  - config_path: ./comic_pipeline.yaml
tools: []

```

```

comic_pipeline.yaml
name: comic_pipeline
agent_class: SequentialAgent
description: A sequential pipeline of agents to create a comic book.
sub_agents:
  - config_path: ./scripting_agent.yaml
  - config_path: ./panelization_agent.yaml
  - config_path: ./image_synthesis_agent.yaml
  - config_path: ./assembly_agent.yaml

```

```
scripting_agent.yamlname: scripting_agent
model: gemini-2.5-pro
agent_class: LlmAgent
description: Narrative & Character Architect.
instruction: >
  You are the Scripting Agent, a Narrative & Character Architect.

  Your input is a seed idea for a comic.

  **Your Logic:**

  1.  **Create a Character Manifest:** You must define exactly 3 specific,
      unchangeable visual identifiers for every character. For example: "Gretel:
      Blue neon hair ribbons, silver apron, glowing boots". This is mandatory.

  2.  **Expand the Narrative:** Expand the seed idea into a coherent narrative
      arc with dialogue.

  **Output:**

  You must output a JSON object containing:

  - "narrative_script": A detailed script with scenes and dialogue.

  - "character_manifest": The mandatory character visual guide.
sub_agents: []
tools: []
```

```
panelization_agent.yamlname: panelization_agent
model: gemini-2.5-pro
agent_class: LlmAgent
description: Cinematographer & Storyboarder.
instruction: >
```

You are the Panelization Agent, a Cinematographer & Storyboarder.

Your input is a narrative script and a character manifest.

****Your Logic:****

1. ****Divide the Script:**** Divide the script into a specific number of panels. The user may define this number, or you should default to 8 panels.
2. ****Define Composition:**** For each panel, you must define a specific composition, camera shot (e.g., "Wide shot", "Close-up"), and the dialogue for that panel.

****Output:****

You must output a JSON object containing a structured list of exactly X panel descriptions, where X is the number of panels. Each item in the list should have "panel_number", "composition_description", and "dialogue".

```
sub_agents: []
```

```
tools: []
```

```
image_synthesis_agent.yaml
name: image_synthesis_agent
model: gemini-2.5-pro
agent_class: LlmAgent
description: Technical Artist & Asset Generator.
instruction: >
  You are the Image Synthesis Agent, a Technical Artist & Asset Generator.

  Your input is a structured list of panel descriptions.

  **Your Logic:**

  1. **Iterate and Generate:** You must iterate through each panel description
  provided in the input. For each panel, you will execute the `generate_image`
  tool.

  2. **Construct Prompts:** For each panel, you will construct a detailed
  prompt for the image generation tool. This prompt must strictly enforce the
  character visual identifiers from the manifest and include the global style:
  "vibrant comic book style, heavy ink lines, cel-shaded, 4k". The prompt must
  also describe the composition and include a request for speech bubbles to
  contain the dialogue.

  3. **Map Output:** You must associate each generated image URL with its
  corresponding panel number and dialogue.

  **Output:**

  You must output a JSON object containing a complete gallery of all generated
  images, mapped to their respective panel data (panel_number, dialogue,
  image_url).
sub_agents: []
tools:
  - name: Agent3.tools.image_generation.generate_image
```

```

assembly_agent.yamlname: assembly_agent
model: gemini-2.5-pro
agent_class: LlmAgent
description: Frontend Developer for comic book assembly.
instruction: >
  You are the Assembly Agent, a Frontend Developer.

  Your input is the mapped gallery of images and panel data.

  **Your Logic:**

  1.  **Generate HTML:** You will write a clean, responsive HTML5/CSS3 file to
  display the comic. The comic must be vertically scrollable, with each panel
  displaying its image on top and the corresponding dialogue or description
  below it.

  2.  **Write File:** You must use the `write_comic_html` tool to save the
  generated HTML to a file named `comic.html` in the `output/` folder.

  3.  Pass the list of image URLs to the tool so it can handle the image assets
  correctly.

  **Output:**

  You will output a confirmation message indicating the path to the final HTML
  file.
sub_agents: []
tools:
  - name: Agent3.tools.file_writer.write_comic_html

```

15. 前往 [ADK \(Agent Development Kit\)](#) 使用者介面分頁，選取「Agent3」 **Agent3**，然後按一下編輯按鈕 (「筆」圖示)。
16. 按一下畫面左下角的「儲存」按鈕。這樣一來，您對主要代理程式所做的所有程式碼變更都會保留。
17. 現在可以開始測試代理程式了！
18. 關閉目前的 [ADK \(Agent Development Kit\)](#) 使用者介面分頁，然後返回 [Cloud Shell 編輯器](#) 分頁。
19. 在 [Cloud Shell 編輯器](#) 分頁的終端機中，先重新啟動 [ADK \(Agent Development Kit\)](#) 伺服器。前往啟動 [ADK \(Agent Development Kit\)](#) 伺服器的終端機，然後按下 **CTRL+C** 鍵關閉伺服器 (如果伺服器仍在執行)。執行下列指令，再次啟動伺服器。

```

#make sure you are in the right folder.
cd ~/adkui

#start the server
adk web

```

20. 按住 Ctrl 鍵並點選網址 (例如 <http://localhost:8000>) 顯示在畫面上。瀏覽器分頁應會顯示 [ADK \(Agent Development Kit\)](#) GUI。
21. 從服務專員清單中選取「Agent3」 **Agent3**。

22. 輸入下列提示詞

Create a Comic Book based on the following story,

Title: The Story of Momotaro

The story of Momotaro (Peach Boy) is one of Japan's most famous and beloved folktales. It is a classic "hero's journey" that emphasizes the virtues of courage, filial piety, and teamwork.

The Miraculous Birth

Long, long ago, in a small village in rural Japan, lived an elderly couple. They were hardworking and kind, but they were sad because they had never been blessed with children.

One morning, while the old woman was washing clothes by the river, she saw a magnificent, giant peach floating downstream. It was larger than any peach she had ever seen. With great effort, she pulled it from the water and brought it home to her husband for their dinner.

As they prepared to cut the fruit open, the peach suddenly split in half on its own. To their astonishment, a healthy, beautiful baby boy stepped out from the pit.

"Don't be afraid," the child said. "The Heavens have sent me to be your son."

Overjoyed, the couple named him Momotaro (Momo meaning peach, and Taro being a common name for an eldest son).

The Call to Adventure

Momotaro grew up to be stronger and kinder than any other boy in the village. During this time, the village lived in fear of the Oni—ogres and demons who lived on a distant island called Onigashima. These Oni would often raid the mainland, stealing treasures and kidnapping villagers.

When Momotaro reached young adulthood, he approached his parents with a request. "I must go to Onigashima," he declared. "I will defeat the Oni and bring back the stolen treasures to help our people."

Though they were worried, his parents were proud. As a parting gift, the old woman prepared Kibi-dango (special millet dumplings), which were said to provide the strength of a hundred men.

Gathering Allies

Momotaro set off on his journey toward the sea. Along the way, he met three distinct animals:

The Spotted Dog: The dog growled at first, but Momotaro offered him one of his Kibi-dango. The dog, tasting the magical dumpling, immediately swore his loyalty.

The Monkey: Further down the road, a monkey joined the group in exchange for a dumpling, though he and the dog bickered constantly.

The Pheasant: Finally, a pheasant flew down from the sky. After receiving a piece of the Kibi-dango, the bird joined the team as their aerial scout.

Momotaro used his leadership to ensure the three animals worked together despite their differences, teaching them that unity was their greatest strength.

The Battle of Onigashima

The group reached the coast, built a boat, and sailed to the dark, craggy shores of Onigashima. The island was guarded by a massive iron gate.

The Pheasant flew over the walls to distract the Oni and peck at their eyes.

The Monkey climbed the walls and unbolted the Great Gate from the inside.

The Dog and Momotaro charged in, using their immense strength to overpower the demons.

The Oni were caught off guard by the coordinated attack. After a fierce battle, the King of the Oni fell to his knees before Momotaro, begging for mercy. He promised to never trouble the villagers again and surrendered all the stolen gold, jewels, and precious silks.

The Triumphant Return

Momotaro and his three companions loaded the treasure onto their boat and returned to the village. The entire town celebrated their homecoming.

Momotaro used the wealth to ensure his elderly parents lived the rest of their lives in comfort and peace. He remained in the village as a legendary protector, and his story was passed down for generations as a reminder that bravery and cooperation can overcome even the greatest evils.

23. 代理程式執行作業時，您可以在 [Cloud Shell 編輯器](#) 的終端機中查看事件。

24. 生成所有圖片可能需要一段時間，請耐心等待或先去喝杯咖啡！圖片生成作業開始後，您應該會看到與故事相關的圖片，如下所示。



圖 34：以連環漫畫形式呈現的桃太郎故事 25。如果一切順利，產生的 HTML 檔案應該會儲存在 HTML 資料夾中。如要改善代理程式，可以返回代理程式助理，要求進行更多變更！

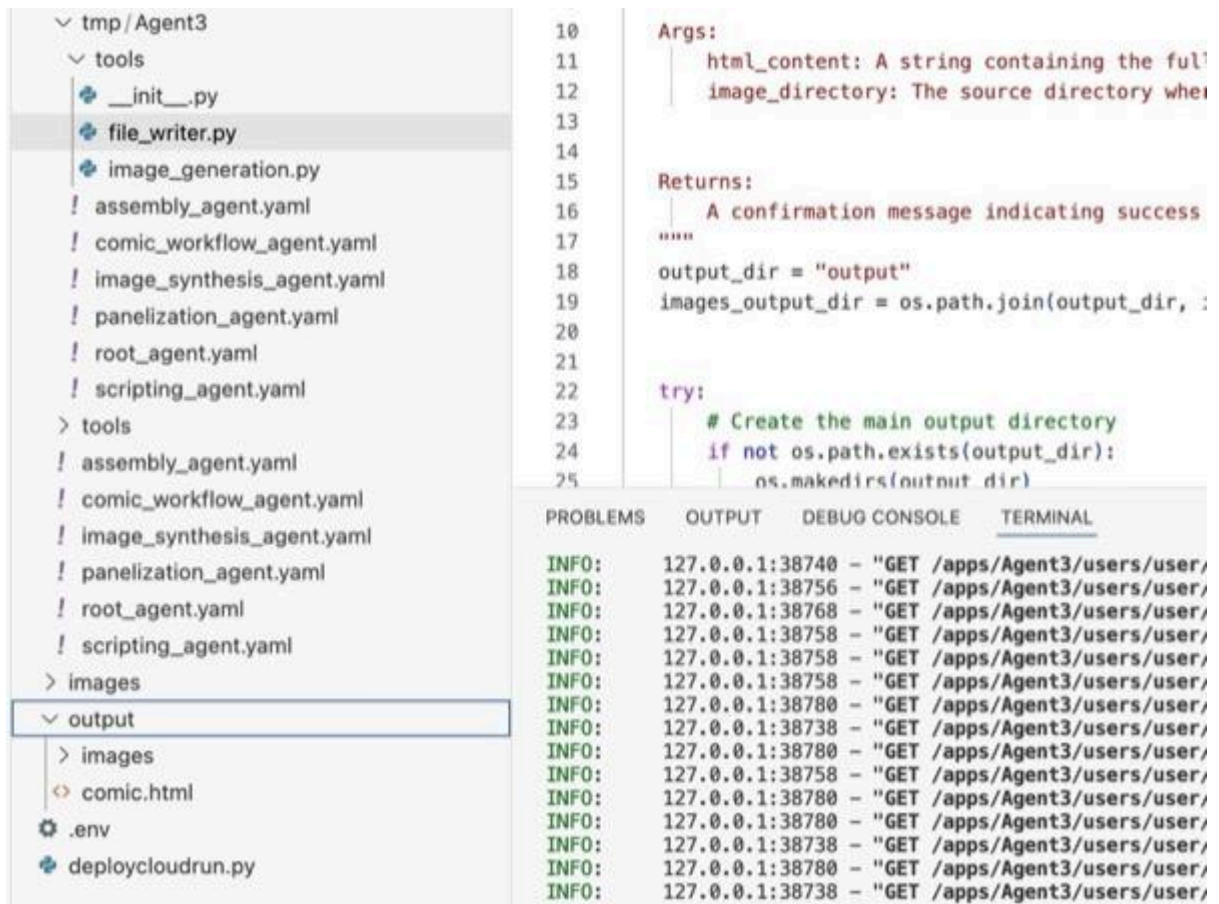


圖 35：輸出資料夾的內容

26. 如果步驟 25 正確執行，您會在 **output** 資料夾中看到 **comic.html**。您可以按照下列步驟進行測試。首先，請依序點選 [Cloud Shell 編輯器](#) 主選單中的「Terminal」>「New Terminal」，開啟新的終端機。這時應該會開啟新的終端機。

```
#go to the project folder
cd ~/adkui

#activate python virtual environment
source .venv/bin/activate

#Go to the output folder
cd ~/adkui/output

#start local web server
python -m http.server 8080
```

27. 按住 Ctrl 鍵並點選「http://0.0.0.0:8080」



圖 36：執行本機網路伺服器

28. 瀏覽器分頁中應會顯示資料夾內容。按一下 HTML 檔案 (例如 comic.html)。漫畫應會如下所示 (您的輸出內容可能略有不同)。

The Story of Momotaro



Momotaro: 'Do not be afraid. The Heavens have sent me to be your son.'



Momotaro: 'I must go to Onigashima. I will defeat the Oni.'

Old Woman: 'Take these, my son. They will give you the strength of a hundred men.'



圖 37：在 localhost 上執行

12. 清除所用資源

現在來清理剛建立的內容。

- 1. 刪除我們剛建立的 **Cloud Run** 應用程式。前往 **Cloud Run**，方法是存取 **Cloud Run**。您應該會看到在上一步建立的應用程式。勾選應用程式旁邊的方塊，然後按一下「刪除」按鈕。

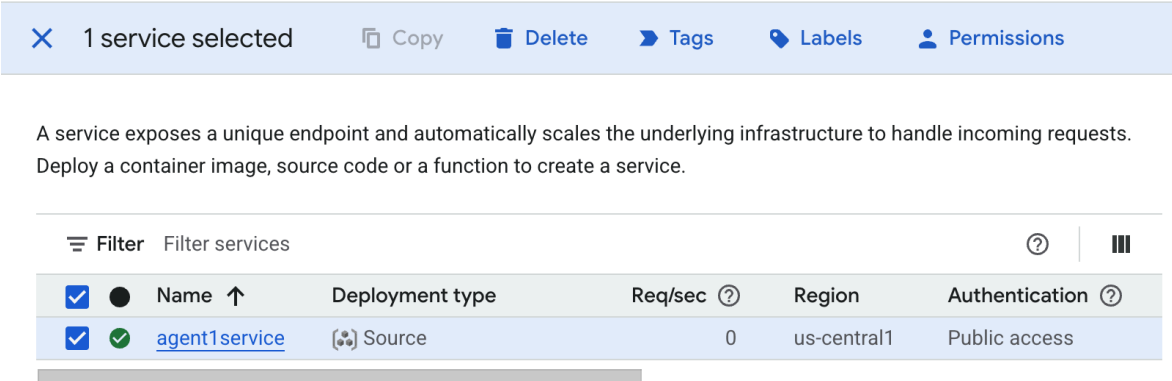


圖 38：刪除 Cloud Run 應用程式 2。刪除 Cloud Shell 中的檔案

```
#Execute the following to delete the files
cd ~
rm -R ~/adkui
```

13. 結論

恭喜！您已使用內建的 [ADK Visual Builder](#) 成功建立 [ADK \(Agent Development Kit\)](#) 代理。您也學會如何將應用程式部署至 [Cloud Run](#)。這項重大成就涵蓋現代雲端原生應用程式的核心生命週期，為您部署複雜的代理程式系統奠定堅實基礎。

重點回顧

在本實驗室中，您學會如何：

- 使用 [ADK Visual Builder](#) 建立多代理應用程式
- 將應用程式部署至 [Cloud Run](#)

實用資源

- [官方 ADK 說明文件](#)
 - [Cloud Run](#)
-